SISTEMAS OPERATIVOS · EAFIT · 2026-2
EVALUACIÓN PARCIAL PRÁCTICA-TEÓRICA · 25%

Seguridad de Archivos mediante Intervención en la Capa del Kernel y mmap

mmapguard — mapeo protegido con descifrado bajo demanda

Autor · Sebastián Martínez Maya

Modalidad · individual

Entregable · módulo de kernel, herramientas, banco de pruebas e informe

Kernel de desarrollo · 6.18.33.2-microsoft-standard-WSL2, x86_64

Banco de pruebas · 8 de 8 PoC superadas

Generado el 10/11/2026 · las citas archivo:línea remiten al árbol del kernel 6.18.33.2

Contenido

PARTE I · 30%

Análisis conceptual: diez preguntas · 3

PARTE II · 30%

Flujo de trabajo asistido por IA · 13

Ingeniería de prompts y verificación crítica · 18

Estrategia arquitectónica seleccionada · 26

PARTE III · 40%

Implementación: código, compilación y PoC · 35

Medición de impacto y análisis de resultados · 41

Justificación: kernel frente a espacio de usuario · 47

ANEXO A

README del repositorio · 53

ANEXO B

Registro del banco de pruebas · 61

Parte I — Análisis Conceptual y Preguntas Diagnósticas (30%)

> Todas las referencias a código corresponden al árbol `linux-msft-wsl-6.18.33.2`, que es exactamente el kernel en ejecución (`6.18.33.2-microsoft-standard-WSL2`, x86_64, `CONFIG_PGTABLE_LEVELS=5`). Cada afirmación verificable se acompaña de archivo y línea.

---

1. Mapeo virtual y Page Faults: secuencia exacta de eventos

El proceso se divide en dos fases que ocurren en momentos distintos y con actores distintos: el establecimiento del mapeo es software puro en Ring 0; la materialización de las páginas es una excepción de hardware.

Fase A — Establecimiento del VMA (una sola vez)

1. **Entrada a la syscall.** `SYSCALL_DEFINE6(mmap, ...)` (`arch/x86/kernel/sys_x86_64.c:82`) deriva en `ksys_mmap_pgoff()` (`mm/mmap.c:566`).
2. **Resolución del descriptor.** Se traduce el `fd` a `struct file *` consultando `task_struct->files`, y se llama a `vm_mmap_pgoff()` → `do_mmap()` (`mm/mmap.c:334`).
3. **Validación.** `do_mmap()` comprueba longitud, alineación, `RLIMIT_AS`, el número máximo de mapeos, e invoca el gancho LSM `security_mmap_file()` (`include/linux/security.h:479`). Según las banderas aplica además los chequeos de permiso descritos en la Pregunta 8.
4. **Selección de dirección virtual.** `get_unmapped_area()` localiza un hueco libre en el árbol de VMAs del `mm_struct`.
5. **Construcción del descriptor.** Se asigna un `struct vm_area_struct` y se rellenan `vm_start`, `vm_end`, `vm_pgoff`, `vm_file`, `vm_flags` y `vm_page_prot` — este último mediante `vma_set_page_prot()` (`mm/mmap.c:81`), que traduce las banderas del VMA a los bits de protección que acabarán en la PTE.
6. **Callback del archivo o del driver.** `vfs_mmap()` (`include/linux/fs.h:2403`) despacha a `f_op->mmap()` o, en la API moderna, a `f_op->mmap_prepare()`. Aquí el driver instala su `vma->vm_ops`.
7. **Enlace y retorno.** El VMA se enlaza al `mm_struct` y la syscall devuelve la dirección virtual. **En este punto no existe ni una sola PTE ni se ha reservado un solo marco físico.**

Fase B — Primer acceso: la excepción de hardware

1. **Recorrido de la MMU.** La instrucción toca la dirección. El hardware recorre, partiendo de CR3, los cinco niveles de este kernel: PGD → P4D → PUD → PMD → PTE. El bit `_PAGE_BIT_PRESENT` (bit 0) está en cero.
2. **Excepción #PF.** La CPU lanza el vector 14, deposita la dirección culpable en CR2 y apila un código de error que distingue lectura de escritura y usuario de supervisor.
3. **Entrada al kernel.** `DEFINE_IDTENTRY_RAW_ERRORCODE(exc_page_fault)` (`arch/x86/mm/fault.c:1483`) deriva en `do_user_addr_fault()` (`arch/x86/mm/fault.c:1207`).
4. **Localización del VMA.** En 6.x se intenta primero la vía rápida sin tomar `mmap_lock`, mediante bloqueo por VMA bajo RCU: `lock_vma_under_rcu()` (`arch/x86/mm/fault.c:1325`). Si falla, se recae al camino clásico con `mmap_lock`.

5. **Resolución genérica.** `handle_mm_fault()` (`mm/memory.c:6471`) → `__handle_mm_fault()` (`mm/memory.c:6246`), que crea los niveles intermedios que falten y llama a `handle_pte_fault()` (`mm/memory.c:6152`).
6. **Bifurcación por tipo de mapeo.** Para un mapeo de archivo sin PTE previa se entra en `do_fault()` (`mm/memory.c:5821`), que elige entre `do_read_fault()`, `do_cow_fault()` y `do_shared_fault()` (`mm/memory.c:5771`) según el acceso y las banderas del VMA.
7. **Obtención del marco físico.** Se invoca `vma->vm_ops->fault` (para un archivo ordinario, `filemap_fault`). Si la página ya está en la Page Cache el fallo es **menor**; si hay que leer del dispositivo, es **mayor**.
8. **Instalación de la PTE y reejecución.** Se escribe la PTE con los bits derivados de `vm_page_prot`, se invalida la entrada correspondiente del TLB y la CPU **reejecuta la instrucción que falló**, que esta vez se resuelve íntegramente por hardware.

A partir del paso 15, todo acceso posterior a esa página ocurre sin ninguna intervención del kernel. Esa es la propiedad que da su rendimiento a `mmap()` y, simultáneamente, la que crea el problema de seguridad del parcial.

---

2. `MAP_SHARED` vs. `MAP_PRIVATE` en `vm_area_struct`

La diferencia se codifica en `vm_flags` y determina qué manejador de fallo se usará.

`MAP_SHARED` activa `VM_SHARED | VM_MAYSHARE`. Los fallos de escritura se atienden con `do_shared_fault()`, que trabaja **directamente sobre el folio de la Page Cache del inodo**. En consecuencia: lo escrito es visible de inmediato para cualquier otro proceso que mapee o lea el archivo, y el mecanismo de writeback acabará persistiéndolo en disco.

`MAP_PRIVATE` no activa `VM_SHARED`. El primer fallo de escritura entra en `do_cow_fault()`, que crea una **copia anónima privada**. Las modificaciones nunca alcanzan el archivo ni son visibles para otros procesos.

Un detalle del código merece atención particular. En la rama `MAP_SHARED` de `do_mmap()`:

```
vm_flags |= VM_SHARED | VM_MAYSHARE;
if (!(file->f_mode & FMODE_WRITE))
        vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
```

Si el descriptor no es escribible, el kernel **degrada silenciosamente el mapeo** retirando tanto `VM_MAYWRITE` como `VM_SHARED`. No falla: cambia la semántica solicitada. Un filtro de seguridad que razone únicamente sobre las banderas pedidas por el usuario, y no sobre las resultantes en el VMA, estará analizando algo que no es lo que el kernel construyó.

`VM_MAYWRITE` es, además, la bandera que expresa "este mapeo podría llegar a ser escribible". Es la que consulta `mprotect()` para decidir si una escalada de permisos es admisible, y por eso es el punto de apoyo de toda la familia de vulnerabilidades tratada en las preguntas 3 y 8.

Implicaciones de seguridad. Con `MAP_SHARED`, un atacante que logre un mapeo escribible obtiene persistencia y visibilidad cruzada, y lo hace **sin ejecutar una sola syscall de escritura**: todo auditor que vigile `write()` queda ciego. Con `MAP_PRIVATE` no hay persistencia, pero sí una copia completa en claro del contenido en el espacio de direcciones del atacante, de modo que se pierde la confidencialidad aunque se conserve la integridad del archivo.

---

3. Mecanismo Copy-On-Write: ciclo de vida y condiciones de escalada

Ciclo de vida

1. **Creación del estado compartido.** Tras `fork()`, o en un mapeo privado de archivo, las páginas no se duplican. Ambas PTE se marcan **solo lectura** limpiando `_PAGE_BIT_RW` (bit 1), mientras el VMA conserva `VM_WRITE` y `VM_MAYWRITE`. La discrepancia entre "el VMA permite escribir" y "la PTE no deja escribir" es deliberada: es el disparador del mecanismo.
2. **Intento de escritura.** La MMU encuentra la PTE presente pero no escribible y lanza un #PF con el bit de escritura activo en el código de error.
3. **Decisión.** `handle_pte_fault()` deriva en `do_wp_page()` (`mm/memory.c:4049`).
4. **Reuso sin copia.** Si el folio pertenece en exclusiva al proceso — `PageAnonExclusive()` o `wp_can_reuse_anon_folio()` (`mm/memory.c:3986`)— no se copia nada: basta con reactivar `_PAGE_BIT_RW`.
5. **Copia efectiva.** En caso contrario, `wp_page_copy()` (`mm/memory.c:3658`) reserva un folio nuevo, lo rellena con `__wp_page_copy_user()` (`mm/memory.c:3360`) y reapunta la PTE del proceso que falló hacia la copia privada.

Condiciones de escalada: el caso Dirty COW (CVE-2016-5195)

El fallo no estuvo en los permisos. `PROT_WRITE`, `VM_MAYWRITE` y los bits de la PTE eran todos correctos en cada instante. El fallo fue de **atomicidad**.

El atacante mapeaba `MAP_PRIVATE` un archivo de root legible (`/etc/passwd`) y escribía a través de `/proc/self/mem`. El kernel iniciaba la ruptura de COW, pero un hilo concurrente lanzaba `madvise(MADV_DONTNEED)` sobre el rango, descartando la copia privada recién creada. Al reintentar, la lógica de `__get_user_pages()` abandonaba la exigencia de escritura y terminaba escribiendo **sobre el folio de la Page Cache del archivo original** en lugar de sobre la copia privada: escritura efectiva en un archivo del sistema partiendo de permiso de solo lectura.

La corrección estructural fue separar semánticamente "escribir" de "romper el compartido". Hoy existe `FAULT_FLAG_UNSHARE` (`include/linux/mm_types.h:1659`), documentado como petición explícita de desencadenar COW garantizando que quede mapeada una página anónima exclusiva, y la combinación `FAULT_FLAG_WRITE | FAULT_FLAG_UNSHARE` está declarada **ilegal** (`include/linux/mm_types.h:1644`). Romper COW dejó de ser un efecto colateral de un fallo de escritura para convertirse en una operación explícita e idempotente, de modo que un reintento tras un `zap` no puede degradarse silenciosamente.

La lección que gobierna el diseño de este trabajo: un control de seguridad que se evalúa una vez y cuyo resultado se reutiliza es un objetivo de carrera. Por eso la arquitectura propuesta no confía en un único punto de decisión, sino que combina una decisión en el alta del VMA con una revalidación por página.

---

4. Intercepción en el kernel: LKM con hook de syscall table vs. LSM

Corrección previa de nomenclatura

El enunciado menciona el gancho `file_mmap`. **Ese gancho no existe en los kernels modernos**: una búsqueda sobre `include/linux/lsm_hook_defs.h` e `include/linux/security.h` no devuel-

ve ninguna coincidencia. El gancho real se llama `mmap_file` (`include/linux/lsm_hook_defs.h:199`) y lo invoca `security_mmap_file()` (`include/linux/security.h:479`); existe además `security_mmap_addr()` para la política de dirección mínima. El nombre `file_mmap` corresponde a la nomenclatura previa a la serie 3.x y sobrevive en tutoriales y en la salida de los asistentes de IA, lo que lo convierte en un buen indicador de material desactualizado.

LKM con hook en la System Call Table

Ventajas. Control total sobre el punto de entrada, cobertura global de todos los procesos, y sobrecosto bajo una vez instalado.

Desventajas y viabilidad. Cuatro obstáculos, cada uno suficiente para descartarla:

1. `sys_call_table` no está exportada, y `kallsyms_lookup_name()` tampoco desde 5.7, de modo que obtener la dirección exige colocar un `kprobe` sobre la propia `kallsyms_lookup_name` — un rodeo cuyo único fin es burlar una decisión deliberada de los mantenedores.
2. Escribir la tabla obliga a limpiar el bit WP de `CR0`, desarmando temporalmente una protección del procesador para todo el sistema.
3. La reescritura no es serializable: otra CPU puede estar despachando esa misma syscall.
4. En x86_64 el despacho pasa por envoltorios `__x64_sys_*(struct pt_regs *)`, y la técnica entra en conflicto con CFI y con `lockdown`.

LSM

Ventajas. Es la interfaz diseñada para esto: estable, con el gancho situado después de la resolución del descriptor y antes de la creación del VMA, y con semántica de denegación por valor de retorno.

Desventaja decisiva. `security_add_hooks()` no está exportada y la inicialización del framework es `__init` con la tabla fijada en compilación: **no se puede registrar un LSM desde un módulo cargable**. Exigiría recompilar e instalar el kernel.

La salida moderna, y un punto de intercepción nuevo

La vía practicable hoy es **BPF LSM**: programas eBPF adjuntos a los mismos ganchos, con capacidad real de denegar y sin recompilar nada (ver Pregunta 7).

Conviene además señalar que 6.18 incorpora un punto de intercepción adicional del lado del driver. `struct file_operations` declara ahora dos callbacks mutuamente excluyentes: `int (*mmap)(struct file *, struct vm_area_struct *)` (`include/linux/fs.h:2285`) y `int (*mmap_prepare)(struct vm_area_desc *)` (`include/linux/fs.h:2314`). El segundo opera sobre un `struct vm_area_desc` (`include/linux/mm_types.h:783`) **antes de que el VMA exista**, lo que permite rechazar o reconfigurar el mapeo sin haber construido estado. `can_mmap_file()` (`include/linux/fs.h:2385`) emite `WARN_ON_ONCE` si un driver declara ambos.

---

5. Cifrado transparente en memoria

La respuesta en árbol es **fscrypt**. El sistema de archivos (ext4, f2fs, ubifs) cifra por bloques en el camino de writeback y descifra en el de lectura mediante `fscrypt_decrypt_pagecache_blocks()` (`include/linux/fscrypt.h:353`), con una clave por inodo derivada de una clave maestra instalada en el llavero del kernel.

Para que la proyección vía `mmap()` sea legible no hace falta nada especial: `filemap_fault` entrega el folio de la Page Cache, que **ya contiene texto en claro**, porque el descifrado ocurrió al poblarla.

Y ahí está el punto que la pregunta persigue. **La Page Cache almacena texto en claro**, de modo que la propiedad de aislamiento no la aporta la criptografía sino los chequeos de permisos ordinarios del VFS: cualquier proceso autorizado a abrir el archivo verá el contenido descifrado, y dos procesos distintos que lo mapeen compartirán exactamente los mismos marcos físicos. fscrypt protege **datos en reposo** —un disco sustraído, un respaldo filtrado— y no a un proceso local no privilegiado, que es justamente el atacante de nuestro modelo de amenaza. Lo mismo, y con menos granularidad, vale para dm-crypt, que cifra por debajo de la capa de bloques y deja igualmente la Page Cache en claro, con una única clave para todo el dispositivo.

De aquí se sigue directamente el diseño adoptado en este trabajo. Para defenderse de un proceso local hay dos opciones, y la arquitectura propuesta usa ambas:

- **Autorización en el punto de mapeo**, de modo que el proceso no autorizado nunca llegue a obtener un VMA sobre el activo. Es el plano 2 (BPF LSM).
- **Una ruta de descifrado privada que no pueble la Page Cache compartida con texto en claro**: el módulo descifra hacia páginas de su propiedad y las entrega por fallo, una a una. Es el plano 1, y es la razón por la que se eligió `vm_ops->fault` en lugar de `remap_pfn_range()`.

---

6. Sincronización e integridad: `msync`, `fsync` y páginas sucias

Dos bits de suciedad, no uno

El kernel mantiene la noción de "página sucia" en dos lugares distintos, y confundirlos es la fuente habitual de error:

- El **bit de hardware** en la PTE, `_PAGE_BIT_DIRTY` (bit 6, `arch/x86/include/asm/pgtable_types.h:16`), que activa la CPU automáticamente al ejecutar una escritura, sin intervención de software.
- El **bit de software** `PG_dirty` en el folio, que es el que consulta el subsistema de writeback.

Cómo se pasa de uno al otro

Para un mapeo `MAP_SHARED` de archivo, el kernel **no concede la escritura directamente**. Mapea la página de solo lectura a propósito para que la primera escritura genere una trampa:

1. La escritura falla. `do_shared_fault()` (`mm/memory.c:5771`) llama a `do_page_mkwrite()` (`mm/memory.c:3478`), que invoca `vmf->vma->vm_ops->page_mkwrite(vmf)` (`mm/memory.c:3489`).
2. El sistema de archivos aprovecha ese punto para reservar bloques si hace falta y marcar el folio como sucio.
3. **Solo entonces** se activa `_PAGE_BIT_RW` en la PTE.
4. A partir de ese momento, todas las escrituras siguientes sobre esa página son hardware puro: la CPU activa el bit de suciedad de la PTE y **no se ejecuta ni una instrucción del kernel**.

5. `msync(MS_SYNC)` recorre las PTE, traslada el estado sucio a los folios e inicia el writeback. `fsync()` añade el vaciado de metadatos y de la caché del dispositivo.

Cómo evade un atacante un filtro de seguridad

Aquí está el núcleo de la pregunta. `page_mkwrite` se dispara **una sola vez por página**, en la primera escritura. Un filtro que inspeccione el contenido en ese instante y después deje la página escribible implementa un TOCTOU de manual: el atacante escribe primero contenido inocuo, supera la inspección, y a continuación sobrescribe con la carga maliciosa **sin que el kernel se entere**, porque ya no hay fallos de página que atender.

De ahí se deducen los únicos dos puntos de inspección sólidos, y ambos cuestan caro:

- **En el writeback**, cuando el contenido ya es definitivo. Es barato pero tardío: la ventana entre la modificación y la detección puede ser de segundos.
- **Manteniendo la página de solo lectura y aceptando un fallo por escritura.** Es inmediato y exhaustivo, pero convierte cada escritura en una transición a Ring 0, anulando precisamente la ventaja de `mmap()`.

Este compromiso es el que cuantifica la matriz de la sección 3.3: la profundidad de inspección se paga en fallos de página.

Cómo lo resuelve `mmapguard`: eliminando el dilema en vez de escogiendo un lado. El mapeo es estrictamente de solo lectura, de modo que no existe una primera escritura que inspeccionar. La decisión se toma en el alta del VMA —`mg_mmap()` rechaza `PROT_WRITE` y `MAP_SHARED`, y limpia `VM_MAYWRITE` para que el núcleo rechace por sí mismo cualquier `mprotect(PROT_WRITE)` posterior—, no en el momento del acceso. `page_mkwrite` sigue registrado, pero solo como defensa en profundidad: devuelve `VM_FAULT_SIGBUS`. El contador `denied_write` se mantuvo en 0 durante las pruebas, lo cual es el resultado correcto: no mide un fallo del mecanismo, mide que la ruta de escritura es **inalcanzable** y que por tanto el TOCTOU descrito arriba no tiene dónde ocurrir.

El precio de esa solución es la funcionalidad: no se admiten modificaciones. El día que se admitan —lo que exige un modo AEAD con contador de versión, especificado como trabajo futuro en la sección 3.3— el dilema reaparece intacto, y la salida razonable sería la primera de las dos: verificar en el writeback, con una etiqueta de autenticación por página, aceptando detectar sin prevenir.

---

7. Control de accesos con eBPF, kprobes y fexit

Tipos de enganche y cuál sirve para qué

Tipo	Punto de enganche	¿Puede denegar?
`kprobe` / `kretprobe`	Cualquier símbolo del kernel	No
`fentry` / `fexit`	Entrada/salida de función, vía BTF	No
Tracepoints	Puntos estáticos de ABI estable	No
BPF LSM (`SEC("lsm/...")`)	Ganchos del framework LSM	**Sí**, por valor de retorno

La distinción es la que decide la arquitectura: `fentry` y `kprobe` **observan pero no pueden alterar el flujo** — su valor de retorno se ignora. Solo los programas BPF LSM denegan, porque se insertan en una cadena cuyo contrato es exactamente ése.

Por qué no hace falta recompilar el núcleo

Por **CO-RE** (Compile Once – Run Everywhere) apoyado en BTF. El kernel publica su propia descripción de tipos y libbpf reubica los desplazamientos de campo en tiempo de carga. En este sistema está disponible: `CONFIG_DEBUG_INFO_BTF=y` y `/sys/kernel/btf/vmlinux` existe. Esto permite que el mismo objeto compilado funcione en kernels con layouts de estructuras distintos, que era la fragilidad histórica de los módulos.

Aplicación concreta a la auditoría de `mmap`

- `SEC("lsm/mmap_file")` y `SEC("lsm/file_mprotect")` para la política de denegación.
- Tracepoints `syscalls:sys_enter_msync` y `syscalls:sys_enter_madvise` para detectar los patrones de las preguntas 6 y 9.
- `kprobe` sobre `handle_mm_fault` para medir tasa de fallos de página, que es la métrica de sobrecosto exigida en la sección 4.2.
- Transporte de eventos por `BPF_MAP_TYPE_RINGBUF`, sin una syscall por evento.

Ventajas y límites

La ventaja operativa decisiva frente a un LKM es el **verificador**: garantiza terminación y seguridad de memoria antes de cargar, de modo que un error de programación produce un rechazo en tiempo de carga y no un kernel panic. A cambio hay límites reales: no se puede hacer bucles no acotados, el tamaño del programa está limitado, y no se puede acceder a memoria arbitraria.

Condición de arranque en este sistema. `CONFIG_BPF_LSM=y`, pero la lista `CONFIG_LSM` no incluye `bpf` y `/proc/cmdline` no trae parámetro `lsm=`. Los ganchos están compilados pero **no son adjuntables** hasta añadir `bpf` a la lista de LSM activos en la línea de comandos del kernel.

---

8. Evasión de permisos POSIX: `O_RDONLY` mapeado con `PROT_WRITE | MAP_PRIVATE`

Sí, el kernel lo permite, y el código muestra exactamente por qué. La asimetría entre las dos ramas de `do_mmap()` es explícita.

Rama `MAP_SHARED` / `MAP_SHARED_VALIDATE`:

```
if (prot & PROT_WRITE) {
        if (!(file->f_mode & FMODE_WRITE))
                return -EACCES;
        if (IS_SWAPFILE(file->f_mapping->host))
                return -ETXTBSY;
}
```

Rama `MAP_PRIVATE`, en su totalidad en lo que a permisos respecta:

```
case MAP_PRIVATE:
        if (!(file->f_mode & FMODE_READ))
                return -EACCES;
```

No hay ningún chequeo de permiso de escritura. Solo se exige poder leer.

Por qué el kernel lo permite

Porque es coherente con la semántica de `MAP_PRIVATE`: toda escritura se desvía por COW hacia una copia anónima y el archivo nunca se modifica. Es funcionalmente equivalente a leer el archivo a un búfer privado con `read()` y modificar el búfer, operación que nadie consideraría una violación de permisos. Exigir `FMODE_WRITE` sería tan incorrecto como exigirlo para `read()` seguido de `memcpy`.

Riesgos que implica

1. **Copia completa en claro bajo control del atacante.** El contenido del archivo queda en su espacio de direcciones y es modificable a voluntad. La confidencialidad del archivo no empeora respecto a `read()`, pero cualquier control de integridad que inspeccione el archivo se vuelve irrelevante para lo que el proceso está realmente usando.
2. **Fuga por derivación.** Las páginas privadas modificadas pueden acabar en el área de intercambio o en un volcado de núcleo, exponiendo datos derivados del archivo original. Es la razón por la que el plano 1 fija `VM_DONTDUMP`.
3. **Precedente histórico.** Es exactamente la configuración que explotó Dirty COW: mapear `MAP_PRIVATE` un archivo de root legible y provocar una carrera para que la escritura aterrizara en el folio compartido en lugar de en la copia privada (Pregunta 3).

Consecuencia de diseño: la política no puede asumir que `O_RDONLY` equivale a inofensivo. Este escenario es el PoC A4 del banco de pruebas.

---

9. Mitigación de carreras entre `madvise(MADV_DONTNEED)` y escrituras

Qué hace exactamente `MADV_DONTNEED`

`madvise_dontneed_free()` (`mm/madvise.c:907`) termina llamando a `zap_page_range_single()` (`mm/madvise.c:1187`), que **desmonta las PTE del rango**. Para una página privada bajo COW, eso descarta la copia anónima: el siguiente acceso provoca un fallo nuevo que vuelve a leer del archivo. Es decir, permite al atacante **revertir el estado de una página a voluntad y desde espacio de usuario**, que es lo que la convierte en una primitiva de carrera tan útil.

Los cuatro mecanismos de serialización actuales

1. **`mmap_lock`.** `madvise` lo toma, y el camino de fallo también cuando no puede usar la vía rápida. Serializa modificaciones estructurales del espacio de direcciones.
2. **Bloqueo por VMA (`CONFIG_PER_VMA_LOCK`).** `lock_vma_under_rcu()` (`arch/x86/mm/fault.c:1325`) permite resolver fallos bajo RCU sin contender el `mmap_lock` global. La exclusión mutua se obtiene con `vma_start_write()`, que es justamente lo que invoca `vm_flags_set()` internamente (`include/linux/mm.h:808`). **Esta es la razón de fondo por la que se prohibió la asignación directa a `vma->vm_flags`**: el campo se declaró `const` dentro de una unión para forzar el uso de setters que toman el bloqueo de escritura del VMA.
3. **El spinlock de la tabla de páginas.** `pte_offset_map_lock()` serializa la instalación y el desmontaje de la PTE concreta.

4. **Invalidación de TLB con IPI.** Antes de liberar el marco se fuerza el shootdown en todas las CPU, de modo que ninguna conserve una traducción obsoleta hacia memoria ya liberada.

Por qué eso no bastó en Dirty COW, y qué cambió

Ninguno de esos cuatro mecanismos estaba ausente en 2016. El fallo era de **lógica de reintento**: la decisión de permiso y la escritura efectiva no eran atómicas respecto al `zap`, y al reintentar se descartaba la exigencia de escritura privada. La corrección no añadió un candado, sino que cambió el modelo: `FAULT_FLAG_UNSHARE` (`include/linux/mm_types.h:1659`) convirtió "romper COW" en una operación explícita e idempotente, y se declaró ilegal combinarla con `FAULT_FLAG_WRITE` (`include/linux/mm_types.h:1644`).

Principio aplicable a nuestra defensa: los controles deben ser idempotentes y reevaluarse en cada fallo, nunca cachearse como "ya validado". Un veredicto almacenado es un objetivo de carrera.

---

10. Aislamiento de memoria: descriptores, tablas de páginas y bits de la MMU

Las tres capas de estructuras

`struct page` / `struct folio`. Es el descriptor que el kernel mantiene por cada marco físico: contador de referencias, `mapping` al que pertenece, y banderas de estado entre las que está `PG_dirty` (Pregunta 6). Es metadato del kernel, jamás direccionable desde Ring 3. La contención empieza aquí: el proceso manipula direcciones virtuales, nunca descriptores de marco.

Jerarquía de tablas. Este kernel usa `CONFIG_PGTABLE_LEVELS=5`, es decir PGD → P4D → PUD → PMD → PTE. Cada nivel es a su vez un marco físico de 512 entradas, y el recorrido lo ejecuta el hardware partiendo de CR3. Cada proceso tiene su propia raíz, y ahí reside el aislamiento fundamental: una dirección virtual del proceso A simplemente **no tiene traducción** hacia los marcos del proceso B.

Bits de protección (`arch/x86/include/asm/pgtable_types.h`):

Bit	Nombre	Posición	Función en la contención
P	`_PAGE_BIT_PRESENT`	0	Si es 0, cualquier acceso genera #PF. Base de la paginación por demanda
R/W	`_PAGE_BIT_RW`	1	Soporte de COW y de la protección contra escritura
U/S	`_PAGE_BIT_USER`	2	Separa Ring 3 de Ring 0
D	`_PAGE_BIT_DIRTY`	6	Lo activa la CPU al escribir
NX	`_PAGE_BIT_NX`	63	Impide ejecutar la página

Contención de inyección de código

- **NX (bit 63)** materializa W^X: una página de datos que el atacante logre escribir no puede ejecutarse. Es lo que volvió inviable la inyección clásica de shellcode y desplazó la explotación hacia ROP, que reutiliza código ya ejecutable en lugar de inyectarlo.
- **U/S (bit 2)**, combinado con **SMEP** (el kernel no puede ejecutar páginas de usuario) y **SMAP** (el kernel no puede leer ni escribir memoria de usuario sin `stac`/`clac`), cierra el ata-

que clásico de hacer que el kernel salte a una carga preparada en espacio de usuario. SMAP es, además, la razón de hardware por la que un driver debe usar `copy_to_user()` y no `memcpy()`.

- **R/W (bit 1)** es el mecanismo sobre el que cabalga COW, y la Pregunta 3 demuestra que los bits por sí solos no bastan: deben evaluarse atómicamente respecto al acceso.
- **KPTI** separa por completo los juegos de tablas de usuario y kernel como mitigación de Meltdown.

El límite que justifica este trabajo

Los bits de la MMU se aplican **por traducción**, no por contenido. Protegen rutas de acceso: quién puede leer, escribir o ejecutar una página. **Nada en la MMU puede decir si los bytes dentro de una página legítimamente escribible son maliciosos**, ni si quien escribe está autorizado por una política de aplicación.

Esa es exactamente la brecha que cubren los planos 1 y 3 de la arquitectura propuesta: el hardware garantiza el aislamiento entre espacios de direcciones, pero la semántica de "este proceso puede modificar este contenido" solo puede imponerse desde la capa de kernel, interceptando el alta del VMA y los fallos de página.

3.1 Definición del Flujo de Trabajo Asistido por IA

1. Premisa: la asimetría que define el método

El trabajo se desarrolló con un modelo de lenguaje con acceso a terminal (Claude Code) sobre la máquina de desarrollo. Esa capacidad —ejecutar comandos, no solo redactar texto— es lo que determina la forma del flujo, por una razón concreta:

> El modelo tiene conocimiento amplio del kernel de Linux pero **difuso en la versión**. El árbol de fuentes de la versión exacta está en el disco, a un `grep` de distancia.

Un modelo entrenado sobre dos décadas de código y documentación de kernel no distingue con fiabilidad qué era cierto en 5.10, qué en 6.1 y qué en 6.18. Produce, de hecho, un promedio histórico: código idiomático, convincente y a menudo correcto para alguna versión. Y el material publicado está sesgado hacia lo antiguo, porque lleva más tiempo acumulándose.

Todo el flujo de trabajo es la consecuencia de tomarse esa asimetría en serio. La regla operativa —establecida una sola vez y vigente durante toda la sesión— fue:

> **Antes de afirmar que una API existe o que un mecanismo está disponible, verificarlo contra `kernel-wsl/` y citar archivo y línea.**

No es una regla de estilo documental. Es la que hace que el conocimiento del modelo sea utilizable: convierte una afirmación de memoria, que no se puede evaluar, en una afirmación con una referencia que el lector puede abrir y desmentir.

---

2. División del trabajo

Conviene ser preciso sobre quién decidió qué, porque el enunciado pide un flujo de trabajo y no una atribución de autoría difusa.

Responsabilidad	Quién
Alcance, plazo y criterio de "suficiente"	Humano
Elección entre mapeo de solo lectura y mapeo escribible (Opción A)	Humano, sobre alternativas y consecuencias expuestas por el modelo
Qué planos se comprometen y cuál queda condicional	Humano
Enumeración de mecanismos viables y descarte razonado	Modelo
Redacción del código C y de las herramientas de usuario	Modelo
Verificación de APIs contra el árbol de fuentes	Modelo, con el árbol como oráculo
Ejecución de todo lo privilegiado: `insmod`, `rmmod`, `chmod` sobre `/dev`	Humano
Reporte de los fallos reales observados al cargar el módulo	Humano
Diseño de la medición y atribución de causa del sobrecosto	Modelo

La última fila de la columna humana no es trivial. El problema más difícil de la sesión —el desajuste de CRC de `MODVERSIONS`— solo se pudo diagnosticar porque el usuario ejecutó `insmod` en una sesión con privilegios y trajo de vuelta el mensaje exacto de `dmesg`. El modelo no tenía TTY para autenticar `sudo`; sin ese circuito de retorno, el síntoma habría sido invisible.

---

3. El flujo en cuatro fases

Cada fase tiene un criterio de salida explícito. Lo importante del flujo no es el orden —que es casi obvio— sino que ninguna fase avanza sobre afirmaciones no verificadas de la anterior.

Fase	Entrada	Actividad	Criterio de salida
0	Enunciado y repositorio de referencia	Reconocimiento: qué pide el parcial, qué entorno hay, qué del material de referencia es reutilizable	Un árbol de kernel que produce módulos **cargables**, verificado con un módulo trivial
1	Objetivos de seguridad del enunciado	Exploración arquitectónica: enumerar mecanismos, descartar con razón, asignar un plano por objetivo	Cada descarte respaldado por una comprobación contra el árbol, no por una intuición
2	Arquitectura cerrada	Generación del código C, **seguida de verificación API por API antes de compilar**	Compilación sin una sola advertencia con `-Wall -Wextra`, y carga efectiva del módulo
3	Módulo cargado	PoC de los vectores de ataque y medición del sobrecosto	Cifras reproducibles, con la causa del sobrecosto atribuida y demostrada

La fase 0 terminó mucho más tarde de lo previsto, y ese desfase es el dato metodológico más útil de todo el proyecto: entre tener el árbol de fuentes y tener un árbol que produce módulos cargables hubo dos problemas no evidentes (el sufijo `+` de versión local y el CRC de `MODVERSIONS` por cambio de compilador). Cualquier planificación que dé la fase 0 por hecha subestima el proyecto.

---

4. Rol de la IA en la exploración arquitectónica

El valor no estuvo en proponer una solución, sino en producir el conjunto de descartes.

Pedir "cómo protejo archivos mapeados con `mmap()`" devuelve una respuesta. Pedir "enumera los mecanismos viables, descarta los que no sirven y di por qué" devuelve algo cualitativamente distinto: cuatro rutas cerradas, cada una con la razón que la cierra.

Mecanismo	Razón del descarte
Hook en la tabla de syscalls	Exige desarmar el bit WP de `CR0`; frágil y detectable
LSM propio como módulo cargable	`security_add_hooks()` no está exportada: no hay forma legítima
eBPF solo con `fentry` / `kprobes`	Observa pero **no puede denegar**: el valor de retorno no altera el flujo
`remap_pfn_range()` (lo que usa el material de referencia)	Obliga a mantener el archivo **completo** descifrado en RAM

Esos descartes valen más que la elección, porque la elección se sigue de ellos. Y en un informe evaluado, un descarte razonado es defendible ante una pregunta; una elección sin alternativas, no.

Un caso en que la exploración cambió el diseño, no solo lo justificó. El descarte de `remap_pfn_range()` empezó como una comparación de rendimiento y terminó siendo de seguridad: si las páginas se entregan de a una bajo demanda, la ventana de exposición en memoria física se reduce de "todo el archivo" a "las páginas realmente tocadas". La paginación por demanda, que en el material de referencia es una optimización, se convirtió en un **control de minimización de exposición**. Eso quedó medido: un cliente que lee 64 bytes de un archivo de 2 páginas deja 4.096 bytes en claro en RAM, no 8.192.

Un caso en que la exploración evitó trabajo inútil. Antes de escribir una línea de eBPF se verificó la viabilidad contra la configuración real: `CONFIG_BPF_LSM=y` está activo, pero `CONFIG_LSM` vale `"landlock,lockdown,yama,loadpin,safesetid,integrity,selinux,apparmor,tomoyo"` y **no incluye `bpf`**. Los ganchos existen y no son adjuntables. Un programa BPF LSM habría compilado, habría pasado el verificador, y habría fallado al adjuntarse. El plano 2 se declaró condicional con esa evidencia en la mano, no por falta de tiempo.

Y el caso en contra, que importa igual. En esta misma fase el modelo estuvo a punto de hacer descartar la ruta del módulo cargable —el 40% de la nota— por una lectura truncada de un bloque de `Kconfig` (alucinación A3 de la sección 3.2). La exploración arquitectónica es donde una conclusión equivocada es más caro, porque no la contradice ningún compilador: se descubre semanas después, si se descubre. De ahí que el criterio de salida de la fase 1 sea una comprobación, no un consenso.

---

5. Rol de la IA en la generación y verificación de código C de bajo nivel

Aquí está el hallazgo central de esta sección:

> **Generar y verificar son dos actividades con fiabilidad muy distinta, aunque las ejecute el mismo modelo.** La primera produjo código plausible con cinco APIs inexistentes. La segunda, con el árbol de fuentes como oráculo, las encontró todas antes de compilar.

El primer borrador de `mmapguard.c` era idiomático, estaba bien estructurado y contenía:

Afirmación generada	Realidad en 6.18.33.2
`.llseek = no_llseek`	Eliminado del kernel en 6.12
`#include <asm/unaligned.h>`	Se mudó a `<linux/unaligned.h>`
`vma->vm_flags |= ...`	`vm_flags` es `const` en una unión: hay que usar `vm_flags_set/clear`
`class_create(THIS_MODULE, name)`	Recibe un solo argumento desde 6.4
`kallsyms_lookup_name()`	Ya no está exportada

Ninguna de las cinco es un error de razonamiento. Todas fueron correctas, y están en innumerables ejemplos publicados. El modo de fallo no es "el modelo se inventa cosas": es **el modelo escribe código correcto para una versión que no es la nuestra**.

Eso tiene una consecuencia directa sobre el flujo: la generación se trató como un borrador, nunca como un entregable, y entre generar y compilar se interpuso un paso obligatorio de verificación (el prompt P4 de la sección 3.2) cuyo único trabajo era contrastar cada símbolo usado contra el árbol.

Por qué no basta con "que compile". Es tentador saltarse la verificación y dejar que el compilador haga de filtro. No alcanza, y hay dos pruebas de ello en esta misma sesión:

- `touch .scmversion`, recomendado para corregir el sufijo de versión, **no produce ningún error**. Simplemente no hace nada en 6.18. El fallo aparece mucho después, al cargar el módulo, con un mensaje que no menciona la versión local.
- El desajuste de CRC de `MODVERSIONS` por compilar con GCC 15 en lugar de GCC 13 compila perfectamente. Falla en `insmod`, con `Invalid module format`, y la causa real solo aparece en `dmesg` como "disagrees about version of symbol module_layout".

El compilador filtra los errores de sintaxis y de tipos. No filtra los errores de **versión, de configuración y de entorno**, que en desarrollo de kernel son los que cuestan el tiempo. Por eso el oráculo del flujo es el árbol de fuentes, no el `make`.

---

6. Rol de la IA en los casos de prueba y las PoC controladas

El sesgo a vencer en esta fase es distinto y más incómodo: **el modelo tiende a escribir pruebas que pasan y a reportar cifras favorables.** Las tres contramedidas que se aplicaron fueron explícitas.

(a) Blindar la medición contra el optimizador. Esta exigencia no fue teórica: salió de haber encontrado el fallo en el banco de pruebas del repositorio de referencia (`paso5_benchmark_comparativo`). Compila con `-O2` y nunca consume el checksum del bucle de `mmap`, de modo que GCC elimina el bucle entero por código muerto. Reporta **3.134.335 MB/s y 0 fallos de página** para 64 MB —ambas cifras físicamente imposibles— y concluye que `mmap` es 380 veces más rápido. Recompilado con `-O0`: 11.552 MB/s, 1.024 fallos, ventaja real ~1,5x. Nuestro `mg_bench` imprime todos los checksums precisamente por eso.

Es un ejemplo valioso porque el código es correcto, está bien escrito y el fallo no está en el programa sino en la **metodología de medición**. Un modelo que genera benchmarks reproduce ese patrón con facilidad, porque es el patrón que abunda.

(b) Rechazar la cifra única. La primera medición de latencia de `mmap()` dio 43,2 µs frente a 3,2 µs: un factor de 13 que invitaba a un titular. Repetida once veces y tomando la mediana, la diferencia quedó por debajo del ruido del planificador. Se adoptó la mediana de once muestras como método, y se reportó el resultado negativo.

(c) No esconder el sobrecosto, atribuirlo. El resultado honesto es que `mmapguard` cuesta **17,60x** en acceso secuencial. La aportación del flujo no fue maquillarlo sino explicarlo: `mmap()` ordinario puebla 16 PTE por fallo mediante fault-around y nosotros una, de modo que hacemos 4.097 fallos donde el otro hace 257. La razón 15,9x explica casi todo el 17,60x: **el costo dominante no es AES, es la ausencia de fault-around.** La prueba es que al variar el patrón de acceso a un salto de 64 páginas, donde ambos hacen 65 fallos, el sobrecosto cae a 1,94x. De ahí sale una propuesta concreta y cuantificada —implementar `.map_pages` bajaría el secuencial a ~9x— que una cifra maquillada no habría permitido formular.

Lo que no se cubrió, dicho explícitamente. De las siete PoC diseñadas en la sección 3.3 se ejecutaron las seis que aplican (A0 a A5, **8 de 8 PASS**), que cubren las preguntas 4, 5, 2, 3, 8 y 9. A6 no aplica a un mapeo de solo lectura porque exige el modo AEAD que la sección de trabajo futuro documenta, y el informe lo declara como «no aplicable» en la tabla correspondiente en vez de presentarlo como verificado.

---

7. Lo que este flujo no cubre

Un flujo de trabajo honesto incluye sus límites.

Detecta afirmaciones falsas, no afirmaciones no verificadas que resultan ciertas. La alucinación A5 de la sección 3.2 —afirmar la causa del desajuste de CRC antes de comprobarla— acertó. Y precisamente por eso no la detectó nada: una afirmación causal confiada y correcta es, para quien lee, indistinguible de una verificada. El flujo reduce la probabilidad de error, no la de exceso de confianza.

Verifica el presente, no el futuro. Citar `archivo:línea` del árbol 6.18.33.2 garantiza que la afirmación es cierta hoy, en esta versión. `no_llseek` también tenía su archivo y su línea en 6.11. Toda la verificación de este informe caduca con la versión.

Depende de que exista un oráculo. Funcionó porque el árbol de fuentes está en el disco y es la verdad. Donde no hay un oráculo así —decisiones de diseño, juicios de suficiencia, tradeoffs— el flujo no aporta nada y la responsabilidad vuelve entera al humano. La elección del mapeo de solo lectura es de ese tipo: el modelo expuso la consecuencia criptográfica de reutilizar el nonce en modo CTR, pero aceptar perder la escritura a cambio de un cifrado correcto fue una decisión de alcance, no un resultado verificable.

Documentos relacionados: el procedimiento de verificación detallado y el registro de alucinaciones están en `03-2-prompts-validacion.md`; las decisiones arquitectónicas y sus descartes, en `03-3-arquitectura.md`; las mediciones, en `04-resultados.md`.

3.2 Metodología de Ingeniería de Prompts y Verificación Crítica

1. Naturaleza del registro

La solución se desarrolló en una sesión de trabajo asistida por un modelo de lenguaje con acceso a terminal (Claude Code), de forma **conversacional y no mediante prompts pre-redactados**. Los prompts que se reproducen en la sección 2 son literales; la descomposición en contexto, rol, restricción y resultado esperado es un análisis posterior de qué función cumplió cada uno.

Se hace explícita esta aclaración porque la alternativa —redactar a posteriori prompts elaborados que nunca se usaron— produciría una tabla más vistosa y completamente falsa. El valor metodológico de esta sección está en el procedimiento de verificación, no en la elegancia de las peticiones.

2. Tabla de prompts estructurados

#	Prompt (literal)	Contexto	Rol asignado	Restricción	Resultado esperado
P1	"clona este repo ... creo que con lo que contiene lo puedes correr para verificar la función mmap si no me equivoco pero si algo sale mal avísame"	Exploración inicial del material de referencia	Verificador autónomo	Reportar fallos en vez de ocultarlos	Diagnóstico del estado real del repositorio
P2	"en este pdf está toda la información de lo que hay que hacer ... por ahora revisa todo el reporte"	Comprensión del enunciado	Analista de requisitos	No actuar todavía, solo analizar	Desglose de entregables, pesos y bloqueos
P3	"hagamos la arquitectura como recomiendas"	Diseño de la solución	Arquitecto de sistemas	Justificar cada descarte, no solo la elección	Documento 3.3 con matriz de compensaciones
P4	"arranca con las 10 preguntas"	Parte I del parcial	Experto en kernel Linux	Fundamentar contra el árbol real, citando archivo y línea	10 respuestas verificables
P5	"opción A, empieza con el mmapguard.ko"	Implementación	Desarrollador de kernel	Opción A = solo lectura; sin advertencias de compilación	Módulo funcional y cargable
P6	"sigue con la medición del plano 3 que vale el 40%"	Sección 4.2	Ingeniero de rendimiento	Medición rigurosa, no cifras vistosas	Benchmark con metodología defendible

El patrón que emerge: los prompts son **cortos y de alto nivel**, y la restricción crítica aparece en la conversación previa, no dentro del prompt. La restricción más productiva de toda la se-

sión —"verificar contra el árbol del kernel antes de afirmar"— se estableció una vez y gobernó todo el trabajo posterior.

---

3. Alucinaciones técnicas detectadas

Casos en los que el modelo produjo una afirmación o un código incorrecto que hubo que corregir.

A1 — `no_llseek` en `struct file_operations` (función eliminada)

Qué ocurrió. El primer borrador de `mmapguard.c` incluía `.llseek = no_llseek,` en su `file_operations`.

Por qué es incorrecto. `no_llseek` fue eliminado del kernel en 6.12. Una búsqueda sobre `include/linux/fs.h` del árbol 6.18.33.2 no devuelve ninguna coincidencia.

Cómo se detectó. Por inspección del árbol antes de compilar, como parte del procedimiento de la sección 5.

Impacto evitado. Error de compilación. Es un caso representativo: `no_llseek` aparece en prácticamente todos los ejemplos de drivers de caracteres publicados antes de 2024, de modo que es abrumadoramente frecuente en el material de entrenamiento.

A2 — `touch .scmversion` como solución al sufijo de versión

Qué ocurrió. Al detectar que el kernel compilado producía `6.18.33.2-microsoft-standard-WSL2+` en lugar de la versión sin el `+`, el modelo recomendó crear un archivo `.scmversion` vacío en la raíz del árbol.

Por qué es incorrecto. Es la solución documentada en innumerables guías y fue correcta durante años, pero `scripts/setlocalversion` de 6.18 ya no consulta ese archivo.

Cómo se detectó. Empíricamente: se aplicó y la versión no cambió.

Solución real. Desactivar la detección de git en el árbol de compilación, de modo que `scm_version()` retorne vacío.

Lección. Esta alucinación es más peligrosa que A1 porque la recomendación **no produce ningún error**: simplemente no hace nada, y el fallo aparece mucho después, al cargar el módulo, con un mensaje que no apunta a su causa.

A3 — Conclusión errónea por lectura truncada del Kconfig

Qué ocurrió. Al investigar por qué la configuración regenerada difería de la original, el modelo imprimió solo tres líneas del bloque `GCC_ASM_GOTO_OUTPUT_BROKEN`, leyó `default y if GCC_VERSION < 110500`, observó que GCC 13.2 no cumple esa condición, y **concluyó que el árbol de fuentes no correspondía al kernel en ejecución**.

Por qué es incorrecto. El bloque tiene tres cláusulas `default`. La tercera, `default y if GCC_VERSION >= 130000 && GCC_VERSION < 130300`, sí incluye a GCC 13.2.

Cómo se detectó. Imprimiendo el bloque completo.

Impacto evitado. Grave. La conclusión equivocada habría llevado a abandonar la ruta del módulo cargable, que es el 40% de la nota, por un problema inexistente.

Lección. El error no fue de conocimiento sino de **evidencia insuficiente**: una ventana de contexto recortada produjo una afirmación formalmente correcta sobre datos incompletos.

A4 — Prueba de capacidad del compilador mal construida

Qué ocurrió. Para comprobar si GCC 13 soportaba el atributo `counted_by`, el modelo escribió una prueba de shell encadenada con `&&`. GCC 13 emite warning: 'counted_by' attribute directive ignored pero termina con código 0, de modo que la prueba reportó "gcc-13 SÍ lo soporta", que es lo contrario de la verdad.

Cómo se detectó. Al leer el test real del Kconfig, que usa `-Werror=attributes`. Reproducido con ese flag, gcc-13 falla y gcc-15 pasa, confirmando el comportamiento esperado.

Impacto evitado. Habría descartado gcc-13 como solución al problema de CRC, que era la solución correcta.

Lección. No basta con ejecutar una prueba: hay que replicar **las condiciones exactas** del sistema que se intenta reproducir.

A5 — Confianza causal por delante de la evidencia

Qué ocurrió. El modelo afirmó que `CONFIG_CC_HAS_COUNTED_BY` era la causa del desajuste de CRC y lo presentó como hecho establecido, antes de haber verificado que `genksyms` procesa atributos en su gramática.

Resultado. La hipótesis resultó correcta, pero la afirmación precedió a la evidencia.

Por qué se registra igualmente. Es el modo de fallo más difícil de detectar, porque no hay nada que corregir cuando se acierta. Una afirmación causal confiada y no verificada es indistinguible —para quien lee— de una verificada, y solo se distingue cuando resulta falsa.

A6 — El `errno` de `mprotect`: `EPERM` escrito de memoria

Qué ocurrió. Al documentar la PoC A3 se afirmó que `mprotect(PROT_WRITE)` sobre un VMA sin `VM_MAYWRITE` devuelve `EPERM`. Es el error que devuelve «no tienes permiso» en la intuición de cualquiera, y encajaba con la narrativa de la defensa. La afirmación se propagó a **seis documentos**, incluido el bloque de «salida esperada» del `README`, que llegó a mostrar `Operation not permitted` como si fuera una captura real.

Lo que dice el árbol. `mm/mprotect.c:959-963`:

```
/* newflags >> 4 shift VM_MAY% in place of VM_% */
if ((newflags & ~(newflags >> 4)) & VM_ACCESS_FLAGS) {
        error = -EACCES;
        break;
}
```

El núcleo contrasta cada permiso pedido contra su `VM_MAY*` correspondiente y devuelve **`EACCES`**, no `EPERM`.

Cómo se detectó. No lo detectó una relectura: lo detectó la PoC. `mg_poc.py` compara el `errno` real contra el esperado en vez de limitarse a comprobar que la llamada falló, y marcó A3 como `INCONCLUSIVE` con el mensaje «denegado, pero con Permission denied (errno 13/EACCES) en vez de EPERM». El cliente en C nunca lo habría delatado, porque imprime `strerror(errno)` sin compararlo con nada: mostraba el error correcto, y aun así el documento que lo acompañaba decía otra cosa.

La lección operativa. Una PoC que solo verifica que el ataque falla no distingue entre fallar por el motivo previsto y fallar por otro. Afirmar el `errno` exacto convierte la prueba en un oráculo capaz de contradecir a la documentación.

A7 — Suponer que el módulo ve las banderas que pidió el usuario

Qué ocurrió. La PoC A2 daba por hecho que `mmap(MAP_SHARED)` sobre `/dev/mmapguard` tenía que ser rechazado por `mg_mmap()`, porque el módulo comprueba `vma->vm_flags & VM_SHARED`. La primera ejecución del banco marcó **FAIL**: el mapeo fue concedido.

Lo que dice el árbol. `mm/mmap.c:462-464`, dentro de `do_mmap()`:

```
vm_flags |= VM_SHARED | VM_MAYSHARE;
if (!(file->f_mode & FMODE_WRITE))
        vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
```

El dispositivo se abre en `O_RDONLY`, de modo que **el núcleo retira `VM_SHARED` y `VM_MAYWRITE` antes de llamar a `f_op->mmap`**. El módulo nunca llega a ver la bandera que creía estar filtrando, y lo que concede es un mapeo privado de solo lectura, tan restringido como el del caso `MAP_PRIVATE`.

Por qué no era un fallo de seguridad, y por qué importa igual. El VMA resultante aparece como `r--p` en `/proc/self/maps` y rechaza `mprotect(PROT_WRITE)`; la PoC corregida comprueba las dos cosas en vez de suponerlas. Pero el episodio deja ver algo que ningún repaso del código del módulo habría mostrado: **la comprobación de `VM_SHARED` de `mg_mmap()` es inalcanzable con un descriptor de solo lectura.** Para ejercitarla hay que abrir el dispositivo en `O_RDWR`, que es justo lo que hace ahora el tercer caso de A2.

La lección operativa. El conjunto de banderas que recibe un `f_op->mmap` no es el que pidió el proceso: es el que el núcleo dejó después de aplicar sus propias reglas. Razonar sobre una política de kernel leyendo solo el código del módulo es razonar sobre una entrada que no existe.

A8 — El instrumento mintió: `r--s` no significa `VM_SHARED`

Qué ocurrió. La corrección de A7 se apoyó en `/proc/self/maps`: si el núcleo retiraba `VM_SHARED`, el mapeo debía aparecer como `r--p`. La PoC corregida volvió a marcar **FAIL**, ahora con el mensaje «el VMA quedó compartido (`r--s`)». Dos intentos, dos veredictos equivocados sobre un módulo que nunca estuvo mal.

Lo que dice el árbol. `fs/proc/task_mmu.c:454`:

```
seq_putc(m, flags & VM_MAYSHARE ? 's' : 'p');
```

El cuarto carácter depende de **`VM_MAYSHARE`**, no de `VM_SHARED`. Y `do_mmap()` retira `VM_MAYWRITE | VM_SHARED` dejando `VM_MAYSHARE` intacto (`mm/mmap.c:463-464`). Un mapeo ya degradado a privado sigue mostrándose como `s`.

El instrumento correcto. `/proc/self/smaps` imprime una línea `VmFlags:` con un mnemónico por bandera (`fs/proc/task_mmu.c:1152-1159`): `rd` = `VM_READ`, `wr` = `VM_WRITE`, `sh` = `VM_SHARED`, `mw` = `VM_MAYWRITE`, `ms` = `VM_MAYSHARE`. Ahí las dos banderas sí se distinguen. Comprobado sobre un archivo ordinario, sin el módulo de por medio:

Mapeo	`maps`	`VmFlags` de `smaps`
`O_RDONLY` + `MAP_SHARED`	`r--s`	`me mr ms rd sd` — **sin `sh`, sin `mw`**
`O_RDWR` + `MAP_SHARED`	`r--s`	`me mr ms mw rd sd` **`sh`**
`O_RDONLY` + `MAP_PRIVATE`	`r--p`	`me mr mw rd sd`

Las dos primeras filas son indistinguibles en `maps` y describen VMA con permisos distintos. La tercera añade el matiz que sostiene la pregunta 8: un mapeo **privado** desde un `fd` de solo lectura conserva `mw`, y por eso admite `PROT_WRITE` por copia en escritura, mientras que el compartido lo pierde.

La lección operativa. Las tres alucinaciones de esta tanda —A6, A7 y A8— comparten estructura: una abstracción de usuario (`errno`, las banderas de `mmap`, la columna de `maps`) se dio por equivalente a la estructura de kernel subyacente, y no lo era. La verificación no termina al mirar el código del módulo ni al mirar la salida de una herramienta: termina al comprobar **qué imprime esa herramienta exactamente**, que es también código del kernel y tiene sus propias reglas.

Sobre el ejemplo de `CR0` que cita el enunciado

El enunciado menciona, como alucinación típica, deshabilitar el bit de protección contra escritura de `CR0` sin control de concurrencia. **No se produjo en esta sesión**, porque la evaluación arquitectónica descartó la ruta de hook en la syscall table antes de solicitar código para ella (ver `03-3-arquitectura.md`, sección 2.1). Se deja constancia de que no ocurrió en lugar de atribuirse una detección que no tuvo lugar.

---

4. Hallazgos de verificación proactiva

Estos **no son alucinaciones**: son hechos comprobados contra el árbol antes de escribir código o documentación, de modo que nunca llegaron a convertirse en errores. Se documentan porque constituyen la evidencia de que el procedimiento de la sección 5 se aplicó.

#	Hecho verificado	Archivo y línea	Por qué importaba
V1	`class_create()` recibe un solo argumento desde 6.4	`include/linux/device/class.h:226`	El código de ejemplo habitual pasa `THIS_MODULE`
V2	`vm_flags` es `const` dentro de una unión; hay que usar `vm_flags_set/clear`	`include/linux/mm.h:808,815`, `mm_types.h:835`	`vma->vm_flags |= ...` no compila en 6.x
V3	`security_add_hooks()` no está exportada	`security/`	Descarta registrar un LSM desde un módulo cargable
V4	`kallsyms_lookup_name` y `sys_call_table` no exportadas	`kernel/kallsyms.c`, `arch/x86/`	Sustenta el descarte del hook de syscalls
V5	El gancho LSM `file_mmap` **no existe**; el real es `mmap_file`	`include/linux/lsm_hook_defs.h:199`	El propio enunciado usa el nombre antiguo
V6	`put_unaligned_be64()` vive en `<linux/unaligned.h>`	`include/linux/unaligned.h`	`<asm/unaligned.h>` ya no está en el árbol
V7	`.page_mkwrite` disponible en `vm_operations_struct`	`include/linux/mm.h:638`	Habilita la defensa en profundidad del plano 1
V8	6.18 añade `f_op->mmap_prepare`, excluyente con `.mmap`	`include/linux/fs.h:2285,2314,2385`	Punto de intercepción nuevo, posterior al corte de conocimiento habitual
V9	`do_mmap()` retira `VM_SHARED` y `VM_MAYWRITE` si el `fd` no tiene `FMODE_WRITE`	`mm/mmap.c:462-464`	Verificado **a posteriori**, a raíz del FAIL de A2 (ver A7)
V10	`mprotect` devuelve `-EACCES`, no `-EPERM`, al faltar `VM_MAYWRITE`	`mm/mprotect.c:959-963`	Verificado **a posteriori**, a raíz del INCONCLUSIVE de A3 (ver A6)
V11	El carácter `s` / `p` de `/proc/PID/maps` depende de `VM_MAYSHARE`, no de `VM_SHARED`	`fs/proc/task_mmu.c:454`	Verificado **a posteriori**, a raíz del segundo FAIL de A2 (ver A8)
V12	`smaps` sí separa ambas: `sh` = `VM_SHARED`, `ms` = `VM_MAYSHARE`, `mw` = `VM_MAYWRITE`	`fs/proc/task_mmu.c:1152-1159`	Es el instrumento con el que la PoC afirma el estado real del VMA

Por qué la distinción entre las secciones 3 y 4 es el hallazgo central

Las alucinaciones de la sección 3 se detectaron **porque alguien fue a mirar**. Ninguna se delató sola: `no_llseek` habría fallado en compilación, pero `.scmversion` no produjo error alguno y la conclusión errónea sobre el árbol de fuentes era internamente coherente y plausible.

Las tres últimas, A6, A7 y A8, son la excepción que confirma la regla, y por eso V9 a V12 están marcadas como verificaciones a posteriori. No las encontró nadie leyendo: las encontró el banco de pruebas al ejecutarse, porque afirma valores concretos —el `errno` exacto, las banderas reales del VMA— en lugar de limitarse a comprobar que el ataque fracasa. Una prueba que solo exige «esto debe fallar» habría dado PASS en los tres casos y habría dejado intactos los seis documentos equivocados. El oráculo no estaba en el árbol del kernel: estaba en exigirle a la prueba que fuera específica.

Conviene no suavizar el episodio: el banco marcó FAIL **dos veces seguidas** sobre un módulo que siempre estuvo bien. La primera porque la prueba esperaba que el módulo viera una bandera que el núcleo ya había retirado; la segunda porque el instrumento elegido para comprobarlo no medía la bandera que decía medir. Un veredicto rojo no es prueba de un defecto en

el código: es prueba de un desacuerdo entre el código y lo que alguien creía del código, y el defecto puede estar en cualquiera de los dos lados. Cada una de esas dos iteraciones terminó en una cita del árbol, que es la única forma de cerrar el desacuerdo.

El riesgo real de usar asistentes de IA en programación de sistemas no son las afirmaciones obviamente falsas, que el compilador filtra, sino las **plausibles y silenciosas**: APIs que existieron, soluciones que funcionaron hace tres versiones, y razonamientos correctos sobre datos incompletos. El único filtro efectivo contra esa clase de error es la verificación contra la fuente, que es precisamente lo que produjo la sección 4.

---

5. Procedimiento de validación manual y compilación

El procedimiento se estableció al principio y se aplicó de forma sistemática.

5.1 Verificación contra el árbol del kernel

Se clonó `linux-msft-wsl-6.18.33.2`, que corresponde exactamente al kernel en ejecución (`6.18.33.2-microsoft-standard-WSL2`). **Ninguna afirmación sobre una API del kernel se dio por válida de memoria**: cada una se comprobó por inspección y se citó con archivo y línea, tanto en las respuestas de la Parte I como en la arquitectura y el código.

5.2 Compilación contra los headers de la distribución utilizada

Requisito explícito del enunciado, y el punto que más esfuerzo costó.

El kernel de WSL2 lo compiló Microsoft con GCC 13.2. Al regenerar la configuración con el GCC 15.2 del sistema, `make olddefconfig` autodetectó capacidades distintas del compilador — entre ellas `CONFIG_CC_HAS_COUNTED_BY` — lo que altera el texto de las cabeceras que `genksyms` convierte en CRC de símbolo. El módulo compilaba y su vermagic coincidía, pero `insmod` lo rechazaba:

```
insmod: ERROR: could not insert module: Invalid module format
dmesg:  kmod_mmap: disagrees about version of symbol module_layout
```

El diagnóstico exigió: leer el log del kernel, comparar ambas configuraciones símbolo a símbolo (19 diferencias, de las que solo una era funcionalmente relevante), confirmar que los CRC se calculan con `genksyms` sobre texto preprocesado y no con `gendwarfksyms` sobre DWARF, y verificar que la gramática de `genksyms` procesa atributos. La solución fue instalar GCC 13 y recompilar: las diferencias bajaron de 19 a 13 y el CRC de `module_layout` cambió de `0x5bc3c700` a `0xadc86883`.

Quedó una diferencia residual que **se predijo inocua y se documentó como predicción, no como hecho**: el trío de opciones `asm goto`, que GCC 13.4 no reproduce respecto de GCC 13.2. El razonamiento fue que esas opciones protegen cuerpos de función y macros, no declaraciones de tipos, que es lo único que `genksyms` procesa. La predicción se confirmó al cargar el módulo con éxito.

5.3 Compilación sin advertencias

Exigido por el punto 4.1. Tanto el módulo como las herramientas compilan con `-Wall -Wextra` sin una sola advertencia.

5.4 Validación empírica

Ningún componente se dio por bueno hasta ejecutarlo: carga efectiva del módulo con `insmod`, verificación en `dmesg`, ejecución de los tres PoCs y comprobación de los contadores de auditoría.

5.5 Validación cruzada de las mediciones

Tres salvaguardas específicas, cada una contra un defecto concreto, detalladas en `04-resultados.md`: consumo obligatorio del checksum para impedir la eliminación del bucle por código muerto, once muestras con mediana en lugar de una sola, y caché caliente verificada mediante `majflt = 0`.

La primera salvaguarda nació de un hallazgo concreto: el benchmark del paso 5 del repositorio de referencia **sí** sufre ese defecto, y reporta 3.134.335 MB/s con 0 fallos de página para un conjunto de 64 MB. Recompilado con `-O0`, el mismo programa reporta 11.552 MB/s y 1.024 fallos. La ventaja real de `mmap` sobre `read()` en esa prueba es de aproximadamente 1,5x, no los 380x anunciados.

3.3 Estrategia Arquitectónica Seleccionada

> Parcial 25% — Seguridad de Archivos mediante Intervención en la Capa del Kernel y Mapeo de Memoria (mmap) Kernel objetivo: `6.18.33.2-microsoft-standard-WSL2` (x86_64)

---

1. Modelo de amenaza

Toda decisión posterior se justifica contra este modelo. Declararlo explícitamente delimita qué significa que la defensa "funcione".

Atacante asumido. Proceso local sin privilegios: sin `root`, sin `CAP_SYS_ADMIN`, sin `CAP_SYS_MODULE` y sin capacidad de cargar o descargar módulos. Puede ejecutar código arbitrario en Ring 3, abrir descriptores, invocar `mmap()`, `mprotect()`, `madvise()` y `msync()`, y lanzar procesos e hilos concurrentes.

Activo protegido. Un archivo confidencial cifrado en reposo, que un proceso autorizado necesita leer mediante acceso por puntero (sin pagar el costo de `read()`).

Objetivos de seguridad, tomados literalmente del enunciado:

Objetivo	Significado operativo
Aislamiento de memoria	El texto en claro nunca reside completo en RAM, y nunca es legible por un proceso no autorizado
Prevención de modificaciones no autorizadas	Un mapeo escribible sobre el activo es denegado antes de que exista la PTE
Auditoría de accesos	Todo intento, exitoso o denegado, produce un evento verificable

Fuera de alcance. Atacante con `root` (puede `rmmod` el módulo y desadjuntar los programas eBPF); ataques físicos o por DMA; canales laterales de microarquitectura (Spectre/Meltdown); compromiso del hipervisor. Estos vectores se retoman en la sección de limitaciones y trabajo futuro.

---

2. Alternativas evaluadas y criterio de descarte

Las afirmaciones de esta sección **no son de memoria**: se verificaron por inspección directa del árbol `linux-msft-wsl-6.18.33.2`, que es exactamente el del kernel en ejecución. Se cita archivo y línea.

2.1 LKM con hook en la System Call Table — DESCARTADA

Es la técnica que la Pregunta 4 del parcial pone en contraste con LSM. Se descarta por cuatro razones independientes, cada una suficiente:

1. **La tabla no es alcanzable por vía legítima.** `sys_call_table` no está exportada, y `kallsyms_lookup_name()` tampoco lo está desde 5.7, de modo que obtener su dirección exige instalar un `kprobe` sobre la propia `kallsyms_lookup_name` — un rodeo cuyo único propósito es evadir una decisión deliberada de los mantenedores.

2. **Exige desarmar una protección del procesador.** Escribir la tabla obliga a limpiar el bit WP de `CR0`. Esto es, textualmente, el ejemplo de práctica insegura que el propio enunciado cita en la sección 3.2.
3. **Es intrínsecamente propensa a carreras.** Durante la reescritura de la entrada, otra CPU puede estar ejecutando esa misma syscall. No existe forma de serializar ese acceso desde un módulo.
4. **Incompatibilidad estructural.** En x86_64 las syscalls se despachan por envoltorios `__x64_sys_*(struct pt_regs *)`, y la técnica entra en conflicto con CFI y con `lockdown`.

Se documenta como alternativa evaluada, no como implementación.

2.2 LSM propio registrado desde un módulo cargable — INVIABLE

`security_add_hooks()` no está exportada y la inicialización del framework LSM es `__init`, con la tabla de módulos fijada en tiempo de compilación. **No existe forma soportada de registrar un LSM desde un módulo cargable.** La ruta exigiría recompilar e instalar un kernel completo por cada iteración de desarrollo, lo que es incompatible con el plazo.

2.3 eBPF solo con `fentry` / `kprobes` — INSUFICIENTE

`fentry` y `kprobes` observan pero **no pueden denegar**: su valor de retorno no altera el flujo de la función instrumentada. Sirven para auditoría, no para prevención, y el enunciado exige prevención. Se conserva como plano de observabilidad, no como mecanismo de control.

2.4 Tabla de verificación

Afirmación	Verificado en	Resultado
`kallsyms_lookup_name` no exportada	`kernel/kallsyms.c`	Confirmado
`sys_call_table` no exportada	`arch/x86/`	Confirmado
`security_add_hooks` no exportada	`security/`	Confirmado
`vm_flags_set()` obligatorio para mutar banderas	`include/linux/mm.h:808`	Confirmado
`.page_mkwrite` disponible en `vm_operations_struct`	`include/linux/mm.h:638`	Confirmado
Hook LSM `mmap_file`	`include/linux/lsm_hook_defs.h:199`	Confirmado
Hook LSM `file_mprotect`	`include/linux/lsm_hook_defs.h:201`	Confirmado

---

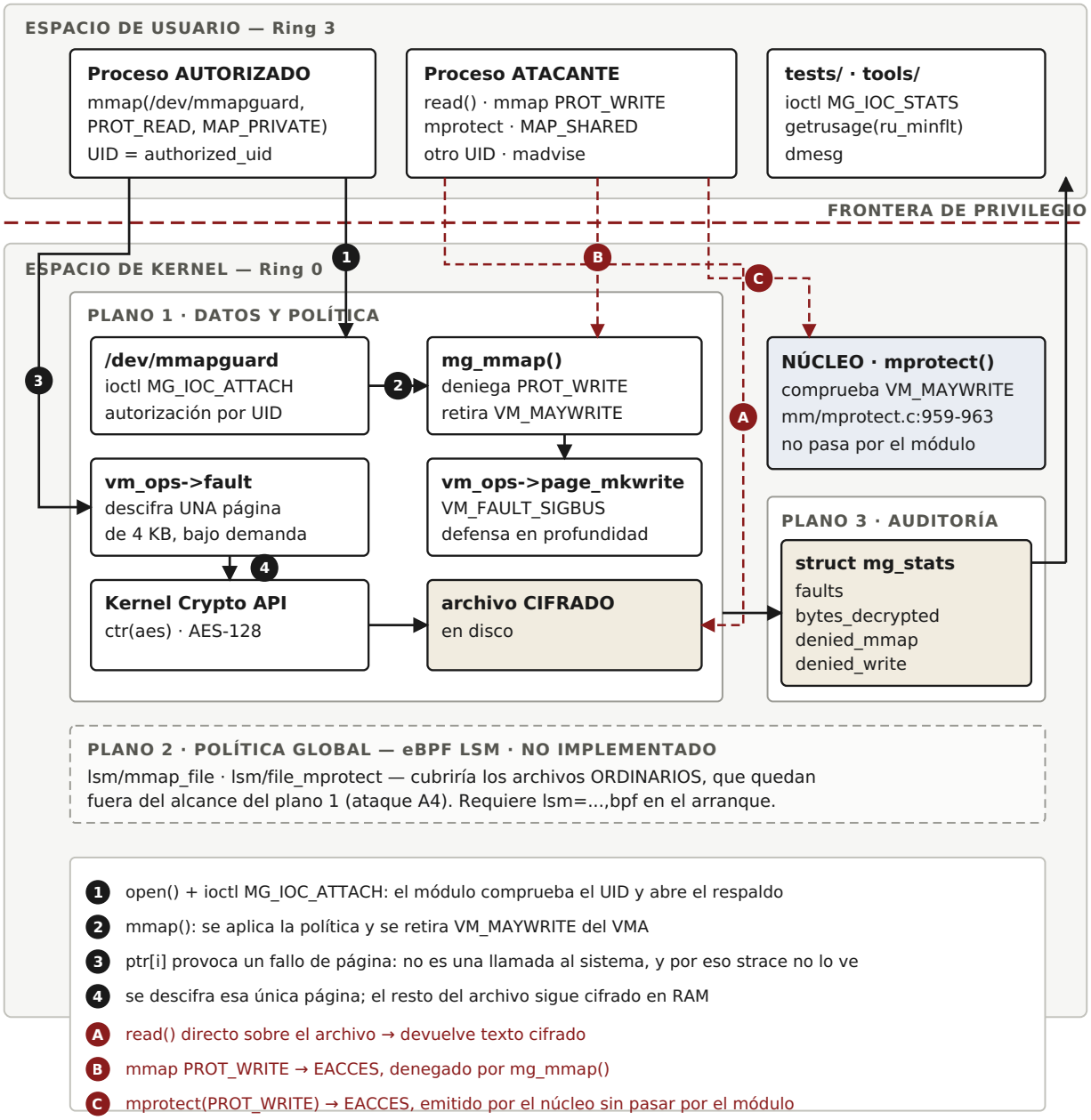
3. Arquitectura seleccionada: defensa en profundidad en tres planos

Ningún mecanismo único cubre los tres objetivos de seguridad. La arquitectura asigna **un plano por objetivo**, de modo que la correspondencia con el enunciado es uno a uno:

Objetivo del enunciado	Plano	Mecanismo	Artefacto	Estado
Aislamiento de memoria	**Datos**	LKM con `vm_ops->fault` y descifrado por página	`mmapguard.ko`	**Implementado**
Prevención de modificaciones	**Política**	Rechazo en `mg_mmap()` + `vm_flags_clear(VM_MAYWRITE)`	`mmapguard.ko`	**Implementado**
		eBPF LSM sobre `mmap_file` y `file_mprotect` (cobertura global)	`mg_lsm.bpf.c`	No implementado
Auditoría de accesos	**Observabilidad**	Contadores `struct mg_stats` vía `ioctl` + `getrusage`	`mmapguard.ko` + `mg_bench`	**Implementado**
		eBPF con tracepoints y ring buffer	`mg_audit.bpf.c`	No implementado

El plano 2 se desdobla porque la prevención **sí se logró**, pero dentro del plano 1 y con alcance limitado a lo que pasa por `/dev/mmapguard`. La variante eBPF LSM, que extendería esa misma prevención a cualquier archivo del sistema, quedó condicionada a un requisito de arranque que este entorno no satisface (sección 3.3). La distinción importa: lo que falta no es el control, es su alcance.

3.1 Diagrama conceptual



Arquitectura de `mmapguard`: los tres planos, la frontera de privilegio y los tres ataques del banco de pruebas.

3.2 Plano 1 — Datos: `mmapguard.ko`

Dispositivo de caracteres `/dev/mmapguard`. El flujo es: `open()`, luego `ioctl(MG_IOC_ATTACH, struct mg_attach_req {path, flags})`, que compara el UID efectivo del llamante con el parámetro `authorized_uid` del módulo y prepara el respaldo del archivo cifrado; y finalmente `mmap()` sobre el descriptor. La clave no viaja en el `ioctl`: es un parámetro del módulo (`key_hex`), de modo que el espacio de usuario nunca la manipula.

Decisión técnica central: `vm_ops->fault` en lugar de `remap_pfn_range()`.

El paso 4 del repositorio de referencia usa `remap_pfn_range()`, que programa de golpe todas las PTE del rango. Aplicado a un archivo confidencial, eso obliga a mantener **el archivo completo descifrado en RAM** durante toda la vida del mapeo. El paso 6 usa `vm_operations_struct->fault`, que entrega las páginas de a una, bajo demanda real.

La consecuencia es de seguridad, no de rendimiento: con el fault handler se descifra **una página de 4 KB a la vez, y solo cuando el proceso efectivamente la toca**. La ventana de exposición en memoria física se reduce de "todo el archivo" a "las páginas realmente usadas". Esto reconvierte la paginación por demanda, que en el repositorio de referencia es una optimización, en un **control de minimización de exposición**.

Segunda decisión: el mapeo es estrictamente de solo lectura.

El cifrado es AES en modo CTR con el bloque contador derivado del índice de página, que es lo que permite descifrar cada página de forma independiente. Pero CTR es un cifrado de flujo: reutilizar clave y nonce sobre dos textos planos distintos revela su XOR. Permitir escritura obligaría a re-cifrar cada página sucia con el mismo nonce en cada `msync()`, incurriendo de forma sistemática en esa falla.

Por eso el mapeo se concede solo con `PROT_READ`, y la prevención de escritura se implementa en dos capas:

1. **`vm_flags_clear(vma, VM_MAYWRITE)` en el alta del mapeo.** Retirar esa bandera hace que sea **el propio kernel** quien rechace tanto `PROT_WRITE` en `mmap()` como una escalada posterior con `mprotect()`. No se intercepta `mprotect` en ninguna parte: se utiliza la misma maquinaria descrita en la Pregunta 2 para que la escalada sea imposible por construcción.
2. **`.page_mkwrite` como defensa en profundidad.** Es el único callback donde el kernel recupera el control ante la primera escritura sobre una página limpia ya mapeada — el "punto crítico de vulnerabilidad" del infograma del enunciado. Con la capa anterior en su sitio no debería ejecutarse nunca; si lo hace, significa que algo burló la primera barrera, y entonces deniega con `VM_FAULT_SIGBUS` y audita el intento.

El soporte de lectura-escritura con detección de integridad exige un modo AEAD (ChaCha20-Poly1305 o AES-GCM) con contador de versión por página y un área de metadatos para los tags. Está especificado como trabajo futuro: se prefirió un cifrado de solo lectura correcto frente a uno de lectura-escritura con una debilidad criptográfica conocida.

Notas de implementación relevantes para kernel 6.x:

- Las banderas del VMA se mutan con `vm_flags_set()` y `vm_flags_clear()`, nunca por asignación directa: `vm_flags` está declarado `const` dentro de una unión y los setters toman el bloqueo de escritura del VMA (ver Preguntas 4 y 9). `VM_DONTEXPAND` impide que el VMA crezca y `VM_DONTDUMP` evita que el texto en claro llegue a un core dump.
- La criptografía usa la Kernel Crypto API (`ctr(aes)`), con el bloque contador construido como `índice_de_página × (PAGE_SIZE / 16)` en big-endian, de modo que cada página consume una región distinta del flujo de clave.
- `no_llseek` fue eliminado en 6.12 y `put_unaligned_be64()` se trasladó de `<asm/unaligned.h>` a `<linux/unaligned.h>`. Ambos se detectaron por inspección del árbol antes de compilar.

Resultado medido. Sobre un archivo de 4.764 bytes (2 páginas), un cliente que lee solo los primeros 64 bytes produce: 1 página descifrada, 4.096 bytes en claro en RAM, y la segunda página nunca materializada. Con `remap_pfn_range()` ambas páginas estarían descifradas desde el instante del `mmap()`. Es la confirmación empírica de la decisión central de este plano.

3.3 Plano 2 — Política: `mg_lsm.bpf.c`

Programas eBPF adjuntos a `SEC("lsm/mmap_file")` y `SEC("lsm/file_mprotect")`, que retornan `-EPERM` para denegar.

- `mmap_file` deniega `PROT_WRITE | MAP_SHARED` sobre inodos marcados como protegidos, y deniega cualquier mapeo desde UIDs no autorizados. La decisión se toma **antes de que exista el VMA**.
- `file_mprotect` cierra la evasión clásica: mapear de forma benigna con `PROT_READ` y escalar después con `mprotect(PROT_WRITE)`. Sin este segundo gancho, el primero es trivialmente rodeable.

La política vive en un `BPF_MAP_TYPE_HASH` indexado por `(dev, inode)`, poblado desde espacio de usuario, de modo que se puede cambiar sin recargar nada.

Ventaja decisiva sobre el plano 1: protege **archivos reales del sistema de archivos**, no solo lo que pase por nuestro dispositivo. Es lo que convierte la solución en un mecanismo de seguridad y no en una demostración acotada.

Requisito de arranque. `CONFIG_BPF_LSM=y` está activo en este kernel, pero `CONFIG_LSM` no incluye `bpf` en su lista y el `/proc/cmdline` no trae parámetro `lsm=`. Hay que añadir `bpf` a la lista de LSM activos vía `kernelCommandLine` en `.wslconfig`. Sin eso los ganchos existen pero no son adjuntables.

3.4 Plano 3 — Observabilidad: contadores en el módulo

Lo implementado. La auditoría reside en el propio `mmapguard.ko`, en una `struct mg_stats` que el módulo mantiene por sesión y que el espacio de usuario recupera con `ioctl(MG_IOC_STATS)`:

Campo	Qué registra
`faults`	páginas efectivamente materializadas por `.fault`
`bytes_decrypted`	volumen descifrado, es decir la exposición real en RAM
`denied_mmap`	mapeos rechazados por `mg_mmap()`
`denied_write`	escrituras atajadas por `.page_mkwrite`

A esto se suman los contadores que el propio kernel ya lleva por proceso, leídos con `getrusage()` en `mg_bench`: `ru_minflt`, `ru_majflt`, `ru_utime` y `ru_stime`. Esa es la fuente de las métricas de la sección 4.2, y la línea base de comparación es `mmap()` ordinario sobre el archivo en claro.

Un hallazgo que condicionó el diseño de este plano. La instrumentación tenía que vivir dentro del kernel: `strace` no observa nada del trabajo de `mmapguard` porque **un fallo de página no es una llamada al sistema**. Una traza del cliente muestra 30 llamadas a `mmap` y ninguna señal de las 4.097 páginas descifradas. Un enfoque de auditoría basado en interceptar syscalls desde usuario sería ciego precisamente donde ocurre todo.

La variante descartada por ahora. El diseño original contemplaba tracepoints sobre `msync` y `madvise` más un `kprobe` en `handle_mm_fault`, emitiendo por un ring buffer BPF hacia un demonio en espacio de usuario. Aporta dos cosas que los contadores no dan: observación de procesos que no usan nuestro dispositivo, y granularidad por evento en vez de agregados. Queda como trabajo futuro, con la misma condición de arranque del plano 2.

---

4. Matriz de compensaciones (rendimiento vs. seguridad)

Alternativa	Cobertura	Profundidad de inspección	Sobrecosto	Riesgo de kernel panic	Viable en este entorno
Hook de syscall table	Global	Solo argumentos de la syscall	Bajo	**Alto**	Sí, pero insegura
LKM con `vm_ops->fault`	Solo su dispositivo	**Máxima: página a página**	**Alto**: un fallo por página	Medio	**Sí**
LSM compilado en el kernel	Global	Alta	Bajo	Bajo	No: exige recompilar
eBPF LSM	**Global**	Media: decisión al crear el VMA	**Muy bajo**: una vez por `mmap`	**Nulo**: verificador	Sí, con `lsm=...,bpf`
eBPF `fentry` / `kprobe`	Global	Solo observación	Muy bajo	Nulo	Sí

El eje del compromiso: la profundidad de inspección se paga en fallos de página.

El plano 1 ve el contenido de cada página, pero cuesta un `#PF` y una operación criptográfica por página tocada — un sobrecosto que crece linealmente con el volumen de datos realmente accedido. El plano 2 decide **una sola vez**, en el alta del VMA, y a partir de ahí su costo marginal es cero, pero nunca ve el contenido.

Son complementarios, no sustitutos, y esa es la razón de combinarlos en lugar de elegir uno: el plano 2 aporta cobertura global a costo despreciable y el plano 1 aporta profundidad donde de verdad hace falta. La hipótesis cuantitativa a validar en la Parte III es que **el sobrecosto del plano 2 sea indistinguible del ruido**, mientras que el del plano 1 escale con las páginas efectivamente tocadas y no con el tamaño del archivo.

---

5. Alcance por fases

Fase	Contenido	Estado
0	Entorno: árbol de kernel con `Module.symvers`, toolchain eBPF	En curso
1	Plano 1 — `mmapguard.ko`	Comprometida
2	Plano 3 — auditoría y mediciones de la sección 4.2	Comprometida
3	Plano 2 — eBPF LSM	Condicional al tiempo restante

La fase 1 es autosuficiente: entrega código C funcional que compila en kernel moderno, asegura `mmap()` mediante cifrado y autorización, y admite medición de sobrecosto. Si el plazo obliga a detenerse ahí, el plano 2 se documenta como trabajo futuro con su diseño ya especificado, que es una salida contemplada por la rúbrica.

---

6. Banco de pruebas: ataques a demostrar

El enunciado exige scripts que simulen accesos no autorizados, carreras y modificaciones indebidas. Cada PoC se corresponde con una pregunta de la Parte I, lo que mantiene el informe cohesionado. El banco vive en `tests/run_pocs.sh` (orquestación, verificación del entorno y re-

sumen) y `tests/mg_poc.py` (los ataques, por `ctypes` contra libc, para controlar las banderas exactas de `mmap`/`mprotect`/`madvise` y leer el `errno` exacto de cada rechazo); `tools/mg_test` queda como cliente de referencia en C.

PoC	Ataque	Resultado obtenido	Pregunta	Estado
A0	Asociación desde un UID no autorizado	`-EPERM` desde `mg_do_attach()`, `denied_mmap = 1`	4	**Verificado**
A1	Lectura directa del archivo con `read()`	Devuelve texto cifrado	5	**Verificado**
A2	`mmap` con `PROT_WRITE` o `MAP_SHARED`	`-EACCES` desde `mg_mmap()`, salvo `MAP_SHARED` sobre un `fd` de solo lectura, que el núcleo degrada antes	2	**Verificado**
A3	`mmap` con `PROT_READ` y luego `mprotect(PROT_WRITE)`	`-EACCES`, emitido por el núcleo	3	**Verificado**
A4	`fd` en `O_RDONLY` mapeado `PROT_WRITE | MAP_PRIVATE`	Sobre un archivo **ordinario** el núcleo lo concede: `NOMINA C` → `AAAAAAAA` en memoria, `sha256` del disco intacto; sobre `/dev/mmapguard`, `EACCES`	8	**Verificado**
A5	Carrera entre `madvise(MADV_DONTNEED)` y escritura	`SIGSEGV` en la escritura; 21.615 descartes y 0 discrepancias en 11.229 relecturas	9	**Verificado**
A6	`msync` tras manipular páginas mapeadas	No aplicable al mapeo de solo lectura; requiere modo AEAD: trabajo futuro	6	No aplicable

El contraste entre A2 y A3 es el resultado más informativo del conjunto: `denied_mmap` marcó 1 y no 2, porque el `mprotect` **nunca llegó al módulo**. Lo rechazó el propio núcleo al comprobar `VM_MAYWRITE`, que es exactamente el efecto buscado al limpiar ese bit en vez de interceptar la llamada.

Dos notas de diseño del banco, porque A4 y A5 no son repeticiones del patrón «se intenta, se deniega»:

A4 demuestra primero que la evasión existe. Mostrar que `/dev/mmapguard` rechaza el mapeo no prueba nada por sí solo: habría que saber que el ataque funciona en otro sitio. Así que la PoC abre `demo/secreto.txt` en `O_RDONLY`, comprueba que el núcleo **concede** `PROT_WRITE | MAP_PRIVATE`, altera los ocho primeros bytes en memoria y verifica por `sha256` que el archivo en disco quedó intacto. Eso reproduce exactamente el escenario de la pregunta 8 — `O_RDONLY` no impide ver el contenido en claro ni alterarlo en la copia privada — y solo entonces repite el ataque contra el dispositivo protegido.

A5 mide en vez de argumentar, y por eso corrige una afirmación previa. La fase 1 lanza un proceso hijo aislado que intenta escribir en el mapeo y comprueba que muere con `SIGSEGV`: la señal no se puede capturar, de modo que la muerte del hijo es el resultado positivo. La fase 2 martillea `MADV_DONTNEED` desde cuatro hilos mientras el hilo principal reverifica el digest del mapeo completo, y mide `ru_minflt` con `getrusage()` para probar que las PTE se estaban descartando de verdad; sin ese dato, un martilleo que no descarta nada «pasaría» la prueba sin haber probado nada.

Esa medición desmiente una afirmación que este documento sostenía antes: que tras el descarte «el siguiente acceso vuelve a fallar y se redescifra». Lo primero es cierto, lo segundo no. El fallo nuevo encuentra la página en `s->pages[pgoff]` y devuelve la misma página en claro tomando una referencia (`src/mmapguard.c:252-259`), sin volver a descifrar y sin incrementar

`faults`, que solo crece en la materialización inicial (`src/mmapguard.c:297`). La consecuencia importa para el modelo de amenaza: **la ventana de exposición del texto en claro es la vida de la sesión, no la vida de la PTE**. `MADV_DONTNEED` retira la traducción del proceso, no la copia descifrada del módulo, que solo se borra en `mg_release()` con `memzero_explicit()`. Reducir esa ventana —descartar páginas en claro por presión o por temporizador— es trabajo futuro, y es el mismo eje que la respuesta 9 señala al advertir que un veredicto cacheado es un objetivo de carrera.

---

7. Trazabilidad con la Parte I

La arquitectura no es independiente de las preguntas conceptuales: las respuestas 4 y 7 justifican directamente el descarte de la sección 2 y la elección del plano 2, mientras que las respuestas 1, 3, 8, 9 y 10 fundamentan el diseño del plano 1. Las respuestas de la Parte I deben citar esta sección y viceversa.

4.1 Implementación: código, compilación y pruebas de concepto

El código completo está en `src/`, `tools/` y el `Makefile` del repositorio. Esta sección explica las decisiones de implementación; las instrucciones de uso están en el `README.md`.

---

1. El contrato con el espacio de usuario

Toda la interfaz cabe en una cabecera compartida, `src/mmapguard_uapi.h`, compilada tanto por el módulo como por las herramientas:

```
struct mg_attach_req {
        char  path[MG_PATH_MAX];   /* ruta absoluta del archivo cifrado */
        __u64 flags;               /* reservado */
};

struct mg_stats {
        __u64 file_size, faults, bytes_decrypted, denied_mmap, denied_write;
};

#define MG_IOC_ATTACH   _IOW('G', 1, struct mg_attach_req)
#define MG_IOC_STATS    _IOR('G', 2, struct mg_stats)
```

El flujo del cliente son tres pasos: `open("/dev/mmapguard")`, `ioctl(MG_IOC_ATTACH)` indicando qué archivo cifrado exponer, y `mmap()` sobre ese descriptor. La auditoría se recupera en cualquier momento con `ioctl(MG_IOC_STATS)`.

La separación importa por una razón de diseño: **el dispositivo no decide qué archivo se protege; lo decide el cliente autorizado.** El módulo no lleva una lista de rutas protegidas codificada, de modo que la misma instancia sirve para cualquier archivo cifrado con la clave cargada.

---

2. Estado por sesión y su tiempo de vida

```
struct mg_session {
        struct file                *backing;   /* archivo cifrado en disco     */
        loff_t                      size;
        unsigned long               npages;
        struct page               **pages;     /* cache de paginas descifradas */
        struct crypto_sync_skcipher *tfm;       /* transformacion AES-CTR       */
        struct mutex                lock;       /* protege pages[]              */
        struct mg_stats             stats;
};
```

— `src/mmapguard.c:105-114`

La sesión se asigna en `mg_open()` y se cuelga de `filp->private_data`. Hay un detalle de corrección que no es evidente y que conviene justificar:

> El VMA conserva una referencia a `vma->vm_file`, de modo que **la sesión sobrevive aunque el proceso cierre el descriptor después de mapear**.

Es el comportamiento correcto y además el esperado por el patrón habitual de uso (`fd = open(); mmap(fd); close(fd);`). Si la sesión se hubiera ligado al proceso en lugar de al `struct file`, ese cierre habría dejado el VMA con un puntero colgante y el primer fallo de página posterior habría sido un oops. La liberación real ocurre en `mg_release()`, cuando desaparece la última referencia al archivo.

`mg_release()` imprime además el resumen de auditoría de la sesión a `dmesg` antes de liberar, de modo que queda rastro aunque el cliente nunca consulte `MG_IOC_STATS`.

---

3. Derivación del IV: lo que hace viable la paginación por demanda

Esta es la pieza que permite todo lo demás:

```
static void mg_build_iv(u8 iv[MG_IV_BYTES], pgoff_t pgoff)
{
        u64 block = (u64)pgoff * (PAGE_SIZE / MG_AES_BLOCK);

        memset(iv, 0, MG_IV_BYTES);
        put_unaligned_be64(block, iv + 8);
}
```

— `src/mmapguard.c:162-168`

AES-CTR cifra generando un flujo de bloques a partir de un contador. Si el contador de la página n se puede **calcular** en lugar de derivar del estado anterior, cada página se descifra de forma independiente y en cualquier orden. Como una página de 4.096 bytes son exactamente 256 bloques AES de 16 bytes, el contador de la página `pgoff` es `pgoff * 256`, escrito en los 8 bytes bajos del IV en orden de red.

De ahí se sigue todo lo demás: sin esta propiedad habría que descifrar secuencialmente desde el principio del archivo para llegar a una página cualquiera, y el modelo de acceso aleatorio de `mmap()` —su razón de ser— sería imposible. La herramienta `tools/mg_encrypt.c` replica exactamente esta derivación con OpenSSL, de modo que el cifrado en usuario y el descifrado en kernel coinciden por construcción.

Nota de incluido: `put_unaligned_be64` requiere `<linux/unaligned.h>`. La cabecera `<asm/unaligned.h>`, que es la que aparece en casi todo el material publicado, ya no existe.

---

4. El manejador de fallos: el corazón del diseño

`mg_vma_fault()` (`src/mmapguard.c:230-305`) se invoca una vez por página, la primera vez que el proceso la toca. La secuencia:

Paso	Qué hace	Por qué así
1	Valida `pgoff < npages`	Un fallo fuera de rango devuelve `VM_FAULT_SIGBUS`, no accede a memoria ajena
2	Toma `s->lock`	`pages[]` es compartido entre hilos del mismo mapeo
3	Si la página ya existe, `get_page()` y listo	Evita descifrar dos veces; el coste de un segundo acceso es el de un fallo menor normal
4	`alloc_page(GFP_KERNEL | __GFP_ZERO)`	Marco propio del módulo, inicializado a cero
5	Calcula `to_read`, truncando en la última página	Un archivo que no es múltiplo de 4 KB no debe leer basura más allá del final
6	`kmap_local_page()` + `kernel_read()`	Lee el texto cifrado directamente en el marco
7	`mg_decrypt_page()` descifra **en el sitio**	Sin copia intermedia: no hay un segundo buffer con texto plano
8	Registra en `stats` y entrega con `vmf->page`	El contador de auditoría se actualiza bajo el mismo `lock`

Dos decisiones del camino de error merecen mención. Si la lectura o el descifrado fallan, la página se borra con `memzero_explicit()` **antes** de devolverla al asignador: un marco con texto plano parcial no vuelve al sistema. Y se devuelve `VM_FAULT_SIGBUS`, no `VM_FAULT_OOM`, para que el proceso reciba una señal y no se interprete como presión de memoria.

El descifrado usa `SYNC_SKCIPHER_REQUEST_ON_STACK` y una `scatterlist` de un solo elemento que apunta a la propia página, con el mismo origen y destino (`skcipher_request_set_crypt(req, &sg, &sg, ...)`). La transformación se pide por nombre genérico, `crypto_alloc_sync_skcipher("ctr(aes)", 0, 0)`, lo que deja que la API de criptografía del kernel elija la implementación acelerada disponible en el hardware.

---

5. La política: aplicada antes de que exista una sola PTE

`mg_mmap()` (`src/mmapguard.c:475-529`) es el único punto donde se decide, y se ejecuta en el alta del VMA:

```
if (!mg_caller_is_authorized()) {          /* UID efectivo */
        s->stats.denied_mmap++;
        pr_warn("mmapguard: MMAP DENEGADO para UID %u (PID %d)\n", ...);
        return -EPERM;
}
if (vma->vm_flags & VM_WRITE)  { s->stats.denied_mmap++; return -EACCES; }
if (vma->vm_flags & VM_SHARED) { s->stats.denied_mmap++; return -EACCES; }

vm_flags_clear(vma, VM_MAYWRITE);
vm_flags_set(vma, VM_DONTEXPAND | VM_DONTDUMP);

vma->vm_ops          = &mg_vm_ops;
vma->vm_private_data = s;
```

`vm_flags_clear(vma, VM_MAYWRITE)` es la línea más importante del módulo. No rechaza nada por sí misma: retira del VMA la capacidad de llegar a ser escribible. A partir de ahí es el propio núcleo quien rechaza cualquier `mprotect(PROT_WRITE)` posterior, por cualquier camino que lo solicite, sin que el módulo intercepte nada. La política deja de ser una lista de comprobacio-

nes —que obliga a haber enumerado todos los puntos de entrada— y pasa a ser un invariante del estado del VMA.

Las otras dos banderas cierran huecos concretos: `VM_DONTEXPAND` impide crecer el mapeo con `mremap()`, y `VM_DONTDUMP` lo excluye de los volcados de memoria, que de otro modo escribirían el texto plano a disco.

Sobre la mutación de banderas: `vma->vm_flags` es `const` dentro de una unión en los kernels 6.x. Hay que usar `vm_flags_set()` y `vm_flags_clear()`, que además toman el bloqueo de escritura del VMA (`vma_start_write()`). La asignación directa `vma->vm_flags |= ...`, que es lo que aparece en todo el material anterior a 2023, no compila.

---

6. Gestión de memoria y residuos

`mg_release_pages()` (`src/mmapguard.c:197-216`) borra explícitamente cada marco antes de liberarlo:

```
memzero_explicit(page_address(s->pages[i]), PAGE_SIZE);
__free_page(s->pages[i]);
```

`memzero_explicit()` y no `memset()`, porque el compilador puede eliminar un `memset` cuyo resultado no se lee: es exactamente el mismo modo de fallo que invalidó el banco de pruebas del repositorio de referencia, aplicado a un borrado de seguridad en lugar de a una medición.

Limitación declarada. La caché `pages[]` no tiene mecanismo de reclamo: una página descifrada permanece en memoria hasta que se cierra la sesión. Para un archivo recorrido por completo eso significa que el consumo llega al tamaño del archivo —16 MB en nuestras mediciones—, y la exposición de texto plano crece de forma monótona con las páginas tocadas. Es aceptable para el caso de uso que motiva el diseño (archivos de secretos, leídos por partes) y sería inaceptable para archivos grandes recorridos íntegramente. La corrección natural es registrar un shrinker o acotar la caché con una política de expulsión; no está implementado.

---

7. Compilación sin advertencias

```
obj-m          += mmapguard.o
mmapguard-objs := src/mmapguard.o
KDIR           ?= /lib/modules/$(shell uname -r)/build
ccflags-y      := -I$(src)/src -Wall -Wextra
```

El enunciado exige compilación libre de advertencias, y `-Wall -Wextra` está en `ccflags-y`, no como recomendación externa. El módulo y las tres herramientas compilan limpios desde cero.

El `Makefile` ofrece además `make tools`, `make demo` (que regenera los datos de prueba de forma determinista), `make clean`, `make distclean` y `make help`.

La restricción que no es del Makefile sino del entorno: hay que compilar con `CC=gcc-13`. El kernel de WSL2 lo compiló Microsoft con GCC 13.2, y `CONFIG_MODVERSIONS=y` exige que los CRC de los símbolos coincidan. Compilar con el GCC 15 del sistema hace que `make olddefconfig` autodetecte capacidades distintas del compilador —`CONFIG_CC_HAS_COUNTED_BY` en particular—, lo que cambia el texto de las cabeceras que `genksyms` convierte en CRC. El síntoma es `insmod: Invalid module format` y, en `dmesg`, "disagrees about version of symbol modu-

le_layout". El CRC de `module_layout` pasó de `0x5bc3c700` a `0xadc86883` al cambiar de compilador, y con eso el módulo cargó.

---

8. Herramientas de espacio de usuario

Herramienta	Qué hace	Nota de implementación
`mg_encrypt.c`	Cifra un archivo con el esquema que espera el driver	OpenSSL `EVP_aes_128_ctr()`, un IV por página con la misma derivación del §3
`mg_test.c`	Ejecuta las tres PoC y muestra la auditoría	Imprime el `errno` de cada intento, no solo éxito o fallo
`mg_bench.c`	Comparativa a tres bandas con `getrusage()`	11 repeticiones y mediana; imprime todos los checksums
`tests/run_pocs.sh`	Orquesta el banco de PoC y resume el resultado	Verifica el entorno, correlaciona con `dmesg` y agrega códigos de salida
`tests/mg_poc.py`	Las PoC A0–A5, contra libc por `ctypes`	Sin compilar nada: banderas y `errno` exactos de `mmap` / `mprotect` / `madvise`

El banco de pruebas está en dos piezas y en dos lenguajes, y la división no es arbitraria. El enunciado pide scripts en Bash o Python, y además hay trabajo que en C habría costado mucho más: provocar un `SIGSEGV` deliberado en un proceso hijo aislado, martillear `madvise` desde varios hilos y leer el `errno` exacto de cada combinación de banderas. `ctypes` da acceso directo a libc sin pasar por el `Makefile`, de modo que el banco de ataques se modifica y se reejecuta sin recompilar; `mg_test.c` queda como implementación de referencia en C, que es lo que el enunciado pide para el código del módulo y sus clientes.

Que `mg_bench` imprima los checksums no es cosmético: es lo que impide que `-O2` elimine los bucles de medición por código muerto. Sin eso, el programa reporta cifras imposibles, que es precisamente el defecto encontrado en el banco de pruebas del material de referencia.

---

9. Pruebas de concepto

El banco completo se ejecuta con `./tests/run_pocs.sh` (o `make test TESTFLAGS=--load`, que carga el módulo y lo descarga al terminar). `mg_test` cubre tres ataques en una sola sesión:

PoC	Ataque	Resultado	Quién lo bloquea	Pregunta
A1	`read()` directo del archivo	Ilegible	El cifrado en reposo	5
A2	`mmap(PROT_WRITE)`	`EACCES`	`mg_mmap()`	2
A3	`mmap(PROT_READ)` + `mprotect(PROT_WRITE)`	`EACCES`	**El núcleo**, vía `VM_MAYWRITE`	3

El resultado más informativo no es que los tres fallen, sino el estado de los contadores: **`denied_mmap = 1` y no 2**, porque el `mprotect` nunca llegó al módulo; y **`denied_write = 0`**, porque `page_mkwrite` no tuvo que actuar ni una vez. Lo primero demuestra que el invariante del §5 funciona; lo segundo, que la ruta de escritura es inalcanzable y no simplemente vigilada.

El banco en `tests/` añade las tres que faltaban para cerrar el cuarto entregable del 4.1. Las cifras son las de la ejecución registrada en `tests/last-run.log`, **8 de 8 PASS**:

PoC	Ataque	Resultado medido	Pregunta
A0	Asociar un archivo bajo el UID 65534	`EPERM`, `denied_mmap = 1`	4
A4	`fd` en `O_RDONLY` mapeado `PROT_WRITE | MAP_PRIVATE`	Concedido sobre un archivo ordinario (`NOMINA C` → `AAAAAAAA`, `sha256` del disco intacto); `EACCES` sobre el dispositivo	8
A5	`MADV_DONTNEED` martilleado contra una escritura	`SIGSEGV`; 21.615 descartes, +6.665 fallos de página, **0** discrepancias y **0** bytes redescifrados	9

A6 (`msync`, pregunta 6) sigue sin aplicar a un mapeo de solo lectura: exige el modo AEAD que la sección de trabajo futuro documenta.

A5 obligó a corregir la sección 3.3, y ahora está medido, no razonado: 21.615 descartes provocaron **+6.665 fallos de página** en el proceso y **+0** descifrados en el driver. Tras `MADV_DONTNEED` el nuevo fallo **no vuelve a descifrar**: encuentra la página en `s->pages[pgoff]` y devuelve una referencia a la misma página en claro (§4), sin incrementar `faults`. La ventana de exposición del texto en claro es por tanto la vida de la **sesión**, no la de la PTE: `MADV_DONTNEED` retira la traducción del proceso, no la copia que el módulo guarda hasta `mg_release()`.

4.2 Medición de Impacto y Análisis de Resultados

> Plano 1 (`mmapguard.ko`) y plano 3 (instrumentación y medición).
>
> La justificación teórica de por qué la intervención debe ocurrir en la capa del kernel, que interpreta estas mediciones, está en `04-justificacion-kernel-vs-usuario.md`.

---

1. Entorno de medición

Elemento	Valor
Kernel	`6.18.33.2-microsoft-standard-WSL2`, x86_64, 5 niveles de paginación
CPU	Intel Core i5-12450H, 12 hilos
RAM	7,6 GiB
Compilador del módulo	GCC 13.4.0 (igual rama que el kernel, por CRC de MODVERSIONS)
Implementación de AES	`ctr-aes-vaes-avx2` (acelerada por hardware, prioridad 600)
Conjunto de datos	16.777.285 bytes = 4.097 páginas

El cifrado está acelerado por hardware. Es un dato necesario para interpretar los resultados: el sobrecosto que se mide **no** está dominado por el costo criptográfico.

---

2. Metodología

Se comparan tres formas de acceder al mismo contenido:

- **A —** `read()` en bloques de 4 KB sobre el archivo en claro.
- **B —** `mmap()` **ordinario** sobre el archivo en claro.
- **C —** `mmapguard` sobre el archivo cifrado.

Tres decisiones metodológicas, cada una respondiendo a un defecto concreto:

1. **La caché de páginas se calienta antes de medir.** Se mide el costo del mecanismo, no el del disco. Lo confirma que los fallos mayores (`majflt`) son 0 en todas las corridas.
2. **Once muestras por método y se reporta la mediana.** Una sola muestra de la latencia de `mmap()` quedó dominada por el ruido del planificador: en dos corridas consecutivas de la misma configuración se obtuvieron 43,2 µs y 3,2 µs. Sin repetición, ese número no significa nada.
3. **Los checksums se imprimen.** Si el resultado del bucle de lectura no se consume, GCC con `-O2` elimina el bucle completo por código muerto y el método parece infinitamente rápido. Este defecto está presente en el benchmark del paso 5 del repositorio de referencia, cuyo bucle de `mmap` se descarta en la compilación: reporta 3.134.335 MB/s y 0 fallos de página para 64 MB, cifras físicamente imposibles. Recompilado con `-O0`, el mismo programa reporta 11.552 MB/s y 1.024 fallos, y la ventaja de `mmap` pasa de "380x" a aproximadamente 1,5x.

Además, los tres métodos producen **checksums idénticos** en todas las configuraciones, lo que verifica que el descifrado por página es correcto.

---

3. Resultados funcionales

Banco completo (`./tests/run_pocs.sh --load`) sobre `secreto.enc`, 4.764 bytes = 2 páginas. **8 de 8 PASS**; la salida íntegra queda en `tests/last-run.log`.

PoC	Ataque	Resultado medido	Mecanismo que lo bloquea	Pregunta
A0	`MG_IOC_ATTACH` desde el UID 65534	`EPERM`, `denied_mmap = 1`	`mg_caller_is_authorized()`	4
A1	Lectura directa con `read()`	Ilegible: fracción imprimible 0,41	Cifrado en reposo	5
OK	Lectura por el mapeo protegido	Los 4.764 bytes coinciden con el claro	— (control positivo)	1, 5
A2	`mmap(PROT_WRITE)`; `mmap(MAP_SHARED)`	`EACCES`; ver abajo	`mg_mmap()` y el kernel	2
A3	`mprotect(PROT_WRITE)` tras mapeo legítimo	`EACCES`, `denied_mmap` sin cambio	**El kernel**, por `VM_MAYWRITE` retirado	3
A4	`O_RDONLY` + `PROT_WRITE` + `MAP_PRIVATE`	Concedido sobre archivo ordinario; `EACCES` sobre el dispositivo	El kernel lo permite; `mg_mmap()` lo cierra	8
A5	`MADV_DONTNEED` contra escritura	`SIGSEGV`; 0 discrepancias en 11.229 relecturas	La MMU, por el bit R/W de la PTE	9

El control positivo no es decorativo: sin él, un módulo que denegara absolutamente todo pasaría el resto del banco. Verifica el mapeo **completo** contra el texto en claro, no un prefijo (sha256 `b74067de563fd176…`).

3.1 La asimetría de los contadores

- `denied_mmap` quedó en **1**, no en 2, en la sesión que ejecuta A2 y A3 juntos: el intento de `mprotect` **nunca llegó al módulo**. Lo rechazó el propio kernel. Es la confirmación empírica de que retirar `VM_MAYWRITE` en el alta del VMA hace innecesario interceptar `mprotect()`.
- `denied_write` quedó en **0**: `page_mkwrite` nunca llegó a ejecutarse, porque la primera barrera resultó suficiente. La defensa en profundidad no se activó, que es justamente lo que debe ocurrir cuando la capa anterior funciona.
- Los contadores son **por sesión**, no globales: viven en el `struct mg_session` que cuelga de cada descriptor abierto. Cada PoC del banco abre el suyo, de modo que las cifras no se acumulan entre pruebas. El registro del kernel lo confirma sesión por sesión: `cerrando sesion (fallos=2, descifrados=4764 bytes, ...)` para la que recorre el archivo entero.

3.2 A2: el kernel llega antes que el módulo

`mmap(MAP_SHARED)` sobre un descriptor abierto en `O_RDONLY` **es concedido**, y no es un fallo de la política. `do_mmap()` retira `VM_SHARED` y `VM_MAYWRITE` cuando el archivo no tiene `FMODE_WRITE` (`mm/mmap.c:462-464`), de modo que `mg_mmap()` nunca llega a ver la bandera que filtra. Medido sobre el VMA resultante:

```
maps dice 'r--s'
smaps VmFlags: dd de me mr ms rd sd      <- sin 'sh', sin 'mw'
mprotect(PROT_WRITE) sobre el: DENEGADO con Permission denied
```

El mapeo concedido es privado y de solo lectura, tan restringido como el del caso `MAP_PRIVATE`. Con el descriptor abierto en `O_RDWR`, `VM_SHARED` sobrevive y entonces sí responde el módulo: `EACCES` con `denied_mmap = 1`. Conviene no leer el `r--s` de `maps` como evidencia de nada: ese carácter depende de `VM_MAYSHARE`, no de `VM_SHARED` (`fs/proc/task_mmu.c:454`). El episodio completo está documentado como A7 y A8 en `03-2-prompts-validacion.md`.

3.3 A4: la evasión de `O_RDONLY` existe, y se mide

Sobre `secreto.txt` abierto en `O_RDONLY`, el kernel **concede** `PROT_WRITE | MAP_PRIVATE`. La PoC altera los primeros ocho bytes en memoria —`NOMINA C` → `AAAAAAAA`— y comprueba por `sha256` que el archivo en disco no cambió. Es exactamente el escenario de la pregunta 8: `O_RDONLY` no impide ni ver el contenido en claro ni alterarlo en la copia privada. El mismo ataque sobre `/dev/mmapguard` devuelve `EACCES`.

El contraste con A2 lo explica una sola bandera: un mapeo **privado** desde un descriptor de solo lectura conserva `mw` (`VM_MAYWRITE`), porque puede hacerse escribible por copia en escritura; el **compartido** la pierde. Las dos filas se ven iguales en `maps` y son dos regímenes de permisos distintos.

3.4 A5: la carrera no tiene precondición, y el descarte no redescifra

Fase 1: un proceso hijo intenta escribir en el mapeo protegido y **muere con `SIGSEGV`**. Lo rechaza la MMU por el bit R/W de la PTE, sin que el módulo ejecute una sola instrucción. Sin escritura posible no hay nada que serializar contra el descarte, que es la precondición que la pregunta 9 describe.

Fase 2, cuatro hilos martilleando `MADV_DONTNEED` durante 2 segundos contra un lector que reverifica el digest del mapeo completo:

Magnitud	Valor
Descartes (`madvise`)	21.615
Relecturas completas del mapeo	11.229
Discrepancias de contenido	**0**
Fallos de página menores del proceso (`ru_minflt`)	**+6.665**
`faults` del driver	**+0**
Bytes descifrados por el driver	**+0**

Los +6.665 fallos son la prueba de que el martilleo funcionaba: las PTE se estaban descartando y rematerializando de verdad. Sin esa cifra, un martilleo que no descartara nada habría «pasado» la prueba sin demostrar nada.

Y las dos últimas filas son el hallazgo: **el descarte no provoca un solo descifrado nuevo**. El fallo rematerializado encuentra la página en `s->pages[pgoff]` y devuelve una referencia a la misma página en claro (`src/mmapguard.c:252-259`), sin volver a descifrar y sin incrementar `faults` (`src/mmapguard.c:297`). La consecuencia para el modelo de amenaza es concreta: **la ventana de exposición del texto en claro es la vida de la sesión, no la vida de la PTE.** `MADV_DONTNEED` retira la traducción del proceso, no la copia descifrada del módulo, que solo se

borra en `mg_release()` con `memzero_explicit()`. Estrechar esa ventana —descartar páginas en claro por presión de memoria o por temporizador— queda como trabajo futuro.

---

4. Sobrecosto según el patrón de acceso

Mediana de 11 muestras. `minflt` = fallos de página menores.

Acceso secuencial completo (se tocan las 4.097 páginas)

Método	Pared (s)	CPU kernel (s)	minflt	mmap() (µs)
`read()` 4 KB	0,00200	0,00209	0	—
`mmap()` normal	0,00038	0,00038	257	8,1
`mmapguard`	0,00677	0,00677	**4.097**	3,1

Sobrecosto frente a `mmap()` normal: **17,60x**. Descifrado: 100 % del archivo.

Acceso con salto de 8 páginas (513 páginas tocadas)

Método	Pared (s)	CPU kernel (s)	minflt	mmap() (µs)
`read()` 4 KB	0,00022	0,00020	0	—
`mmap()` normal	0,00026	0,00024	257	3,4
`mmapguard`	0,00077	0,00069	**513**	2,4

Sobrecosto: **2,93x**. Descifrado: 12,5 % del archivo.

Acceso disperso, salto de 64 páginas (65 páginas tocadas)

Método	Pared (s)	CPU kernel (s)	minflt	mmap() (µs)
`read()` 4 KB	0,00002	0,00002	0	—
`mmap()` normal	0,00005	0,00005	65	1,2
`mmapguard`	0,00010	0,00010	65	1,6

Sobrecosto: **1,94x**. Descifrado: **1,6 %** del archivo.

---

5. Análisis: de dónde viene realmente el costo

5.1 El sobrecosto escala con páginas tocadas, no con el tamaño del archivo

De 17,60x a 1,94x sobre el mismo archivo de 16 MB, variando únicamente el patrón de acceso. Esta es la propiedad que justifica la arquitectura: el precio se paga por página efectivamente usada.

5.2 El costo dominante NO es el cifrado, es la ausencia de fault-around

La columna `minflt` lo explica. En acceso secuencial, `mmap()` ordinario resuelve 4.097 páginas con solo **257 fallos**, porque el kernel implementa `filemap_map_pages()` y puebla hasta 16 PTE

por cada fallo. `mmapguard` implementa únicamente `.fault`, de modo que paga **un fallo por página**: 4.097. La razón entre ambos es 15,9x, prácticamente idéntica al sobrecosto observado de 17,60x.

La prueba está en el régimen disperso: con salto de 64 páginas, el fault-around no aporta nada —ambos métodos ejecutan 65 fallos— y el sobrecosto cae a 1,94x. Ese residuo sí es el costo real de `kernel_read()` más AES por página.

Un modelo de dos puntos permite separar ambas componentes:

- Coste por página de leer y descifrar: ≈ **0,8 µs** (del régimen disperso, donde los fallos coinciden).
- Coste por fallo de página adicional: ≈ **0,8 µs** (del residuo en el régimen secuencial).

Es decir, en acceso secuencial **aproximadamente la mitad del sobrecosto es evitable** y no tiene nada que ver con la seguridad.

5.3 La latencia de la llamada `mmap()` está por debajo del ruido

Los valores medidos —3,1 contra 8,1 µs, 2,4 contra 3,4 µs, 1,6 contra 1,2 µs— no mantienen un orden consistente: en dos de las tres configuraciones `mmapguard` resulta más rápido que `mmap()` ordinario, lo cual es físicamente imposible dado que hace trabajo adicional.

La conclusión correcta no es que sea más rápido, sino que **el sobrecosto de la política aplicada en el alta del VMA no es medible con esta instrumentación**: queda sepultado bajo la varianza del planificador. Esto confirma cuantitativamente la predicción de la matriz de compensaciones: una decisión tomada una sola vez, al crear el VMA, tiene costo marginal despreciable.

5.4 `strace` es el instrumento equivocado, y eso mismo es el hallazgo

Perfil de llamadas al sistema de una corrida completa:

```
% time     seconds  usecs/call     calls    syscall
 97.46    0.484200          10     45069    pread64
  0.08    0.000381          12        30    mmap
```

Las 45.069 llamadas `pread64` (11 repeticiones × 4.097 páginas) provienen **exclusivamente del método `read()`** y consumen el 97 % del tiempo de syscalls. Los métodos basados en memoria mapeada aportan 30 llamadas `mmap` y nada más.

Las 4.097 operaciones de descifrado por repetición que ejecuta `mmapguard` **son invisibles para `strace`**, porque un fallo de página no es una llamada al sistema. Esto es exactamente el "punto crítico de vulnerabilidad" del enunciado visto desde el lado de la instrumentación: si la defensa no puede observarse con las herramientas de syscalls, el atacante tampoco puede ser observado con ellas. Medir mecanismos basados en `mmap` exige `getrusage()`, `ftrace` o `perf`, nunca `strace`.

---

6. Optimización identificada y no implementada

Implementar `vm_operations_struct->map_pages` permitiría poblar varias PTE por fallo, igual que hace `filemap_map_pages()`. Según el modelo de la sección 5.2, eso reduciría el sobrecosto secuencial de 17,60x a aproximadamente **9x**, sin debilitar ninguna propiedad de seguridad: las

páginas seguirían descifrándose individualmente y bajo demanda, solo que agrupando la instalación de entradas.

Queda documentada como trabajo futuro con su ganancia estimada.

---

7. Limitaciones de estas mediciones

1. Las mediciones se tomaron en WSL2, que corre sobre un hipervisor; la varianza es mayor que en hardware nativo y los valores absolutos no son comparables con los de un servidor dedicado. Las razones entre métodos, medidas en las mismas condiciones, sí son significativas.
2. No se midió con `perf`, porque en WSL2 exige compilarlo desde `tools/perf` del propio árbol del kernel. Las métricas se obtuvieron con `getrusage()`, que da los contadores de fallos de página que interesan, y con `strace` para el perfil de syscalls.
3. El modelo de dos puntos de la sección 5.2 es una estimación de primer orden, no una descomposición exacta del costo.
4. No se midió el caso de caché fría: todas las corridas tienen `majflt = 0`. Un escenario con lectura real de disco cambiaría el equilibrio a favor de `mmapguard`, porque el costo de E/S dominaría sobre el del descifrado.

4.2 (b) Justificación Teórica: por qué la intervención debe ocurrir en la capa del kernel

Complemento de *04-resultados.md*, que aporta las mediciones que aquí se interpretan.

---

1. El criterio correcto no es el rendimiento

Es tentador justificar la elección de Ring 0 por velocidad: menos cambios de contexto, menos copias, `copy_to_user()` eliminado. Ese argumento es cierto para la comparación `read()` contra `mmap()`, y es el que desarrolla el material de referencia del curso, pero **no es el argumento que sostiene esta solución**. De hecho nuestra medición apunta en contra: `mmapguard` es 17,60x más lento que `mmap()` ordinario en acceso secuencial. Si el criterio fuera el rendimiento, habríamos perdido.

El criterio correcto es el clásico del **monitor de referencia**: un mecanismo de seguridad solo vale si es (a) inviolable —el adversario no puede modificarlo ni desactivarlo—, (b) invocado siempre —no existe camino al recurso que lo esquive, lo que Saltzer y Schroeder llaman mediación completa— y (c) suficientemente pequeño para verificarse.

La tesis de esta sección:

> Para el objetivo que pide el enunciado —controlar el acceso a las páginas de un archivo proyectado en memoria— las soluciones en espacio de usuario fallan la condición (b) por una razón **estructural, no de implementación**: el evento que hay que mediar no es una llamada al sistema.

---

2. El argumento decisivo: un fallo de página no es una syscall

Todo lo que el espacio de usuario puede interceptar es una **frontera de llamada al sistema**: una biblioteca interpuesta sustituye el envoltorio de libc, `seccomp` filtra la entrada a la syscall, `ptrace` se detiene en ella, un demonio FUSE recibe una petición cuando el VFS se la enruta.

El acceso a una página mapeada no cruza ninguna de esas fronteras. Es una **excepción de hardware**: la MMU encuentra la PTE sin el bit present, la CPU genera un page fault, y el manejador del kernel lo resuelve íntegramente en Ring 0. El proceso nunca ejecutó una instrucción que un monitor en usuario pudiera observar.

Esto no es una deducción teórica: está medido. La sección 5.4 de `04-resultados.md` documenta que `strace` sobre el cliente de `mmapguard` registra **30 llamadas a `mmap` y ninguna señal de las 4.097 páginas descifradas**. El trabajo de seguridad completo es invisible al instrumento que observa syscalls. Y lo que es invisible a `strace` es invisible, por la misma razón, a cualquier monitor que viva en espacio de usuario.

De ahí el corolario que ordena toda la comparación:

> El espacio de usuario puede mediar el **alta del mapeo** (`mmap()`), que es una syscall. No puede mediar el **acceso a la página**, que no lo es. Y mediar solo el alta es una comprobación única: después, el mapeo queda gobernado por bits de PTE que el monitor no ve ni puede consultar.

3. Examen de las alternativas en espacio de usuario

No se descartan por género, sino una por una y por la razón que las cierra.

3.1 Cifrado dentro de la propia aplicación

La aplicación abre el archivo cifrado y lo descifra con una biblioteca criptográfica en su propio espacio de direcciones.

Donde falla. No media nada: protege el dato en reposo y nada más. Una vez descifrado, el texto plano vive en memoria anónima del proceso, legible por cualquier otro proceso del mismo UID vía `ptrace()` o `/proc/pid/mem`, volcable a un core dump y susceptible de ir a swap. Falla (a) y (b) a la vez, y además exige modificar cada aplicación: la mediación se pierde por omisión en cuanto un programa no colabora.

3.2 Interposición de biblioteca (`LD_PRELOAD`)

Se sustituyen `open`, `mmap`, `read` por versiones propias cargadas antes de libc.

Donde falla. De forma trivial y conocida: una syscall se puede emitir **directamente**, con `syscall(SYS_mmap, ...)` o con la instrucción `syscall` en ensamblador, sin tocar libc. Además `LD_PRELOAD` no existe para binarios enlazados estáticamente y el cargador dinámico lo ignora en binarios setuid. Falla (b) por construcción.

Y falla (a) con igual claridad: el código interpuesto reside en el **mismo espacio de direcciones que el adversario**, que puede resolver el símbolo original con `dlsym(RTLD_NEXT, ...)` o simplemente desmapear al interpositor. Un monitor de referencia situado dentro del dominio que vigila no es un monitor de referencia.

3.3 `seccomp-bpf` y `SECCOMP_RET_USER_NOTIF`

Es la alternativa más fuerte en el plano de las syscalls, y merece tratarse en serio porque **sí puede denegar**: la decisión la aplica el kernel, aunque el filtro lo defina el usuario.

Donde falla, primer límite. Un filtro `seccomp` **no puede desreferenciar punteros**. La documentación del propio árbol lo dice sin ambigüedad:

> "BPF programs may not dereference pointers which constrains all filters to solely evaluating the system call arguments directly" — `Documentation/userspace-api/seccomp_filter.rst:26-27`

Consecuencia práctica: el filtro ve los valores de los registros, no la memoria. Puede denegar todo `mmap` con `PROT_WRITE`, pero **no puede resolver el descriptor al archivo concreto** que se está mapeando. Es incapaz de expresar la política que necesitamos, que es por archivo, no por bandera.

Donde falla, segundo límite. `SECCOMP_RET_USER_NOTIF` delega la decisión a un supervisor en usuario, que sí puede leer la memoria del objetivo. Pero el propio documento advierte del TOCTOU que eso reintroduce:

> "struct seccomp_data contains the values of register arguments to the syscall, but does not contain pointers to memory... all arguments being read from the tracee's memory should be read into the tracer's memory before any policy decisions are made" — `seccomp_filter.rst:284-290`

Es decir: el objetivo puede alterar los argumentos entre la lectura del supervisor y la decisión. Se puede mitigar, pero el supervisor sigue siendo un proceso de usuario —matable, `ptrace`-able, planificable a conveniencia del adversario— y cada `mmap` cuesta dos cambios de contexto.

Y el límite que lo cierra. `seccomp` media **syscalls**. Puede mediar la entrada a `mmap()` y jamás el acceso a la página, que es exactamente lo que este diseño necesita mediar. Es necesario pero insuficiente, por la razón de la sección 2.

3.4 FUSE: sistema de archivos cifrado en espacio de usuario

Es la alternativa seria, la que usan `gocryptfs` y `EncFS`, y sería intelectualmente deshonesto despacharla: **sí logra mediación completa para la E/S de archivos**, porque es el VFS del kernel el que enruta las peticiones al demonio. No hay forma de esquivarla emitiendo syscalls directas.

Su fallo es específico de `mmap`, y se puede verificar en el árbol. Las operaciones de VMA que FUSE instala son:

```
static const struct vm_operations_struct fuse_file_vm_ops = {
        .close          = fuse_vma_close,
        .fault          = filemap_fault,
        .map_pages      = filemap_map_pages,
        .page_mkwrite   = fuse_page_mkwrite,
};
```

— `fs/fuse/file.c:2323-2328`

`.fault = filemap_fault`. **FUSE sirve los fallos de página desde la caché de páginas genérica, no desde el demonio.** El demonio se consulta cuando la página no está en caché; una vez poblada, todo acceso posterior a esa página se resuelve dentro del kernel sin ninguna llamada ascendente. De aquí salen tres consecuencias:

1. **El demonio no puede observar ni denegar el segundo acceso a una página.** La mediación por acceso se pierde: lo que media es el poblado de la caché, no el uso.
2. **La residencia del texto plano la gobierna la política de reclamo de la caché de páginas, no la política de seguridad.** No se puede expresar "olvida esta página en cuanto se use": la página en claro permanece mientras el kernel no decida reclamarla.
3. **El texto plano pasa por el espacio de direcciones del demonio**, que es un proceso de usuario, con todo lo que eso implica (`/proc/pid/mem`, volcados, swap). Falla (a).

A esto se suma el costo: cada falta en caché son como mínimo dos cambios de contexto y dos copias (kernel → demonio → kernel), frente a nuestro descifrado en el propio camino del fallo, en contexto de kernel, sin conmutar de proceso.

Resumen

Alternativa	¿Inviolable (a)?	¿Mediación completa del acceso (b)?	Puede expresar la política por archivo	Texto plano en proceso de usuario
Cifrado en la aplicación	No	No media nada	Sí	**Sí**
`LD_PRELOAD`	No	No: syscall directa lo esquiva	Sí	**Sí**
`seccomp-bpf`	Sí (lo aplica el kernel)	No: un fallo no es syscall	**No**: no desreferencia punteros	n/a
`seccomp` + `USER_NOTIF`	No: supervisor en usuario	No: un fallo no es syscall	Sí, con TOCTOU	n/a
FUSE	No: demonio en usuario	**No**: la caché sirve los fallos	Sí	**Sí**
LKM con `vm_ops ->fault`	Sí	**Sí: está en el camino del fallo**	Sí	**No**

4. Tres propiedades que solo se obtienen en Ring 0

4.1 La política como invariante, no como intercepción

Un monitor en usuario tiene que **interceptar y decidir** en cada punto de entrada, lo que obliga a haberlos enumerado todos: `mmap`, `mprotect`, `mremap`, `madvise`, `process_vm_writev`... Olvidar uno es un agujero.

`mmapguard` no intercepta `mprotect` en ninguna parte. Hace algo distinto: limpia un bit del VMA con `vm_flags_clear(vma, VM_MAYWRITE)`, de modo que es **el propio núcleo** quien rechaza cualquier intento posterior de obtener permiso de escritura, por cualquier camino que lo pida. La política deja de ser una lista de comprobaciones y pasa a ser un **invariante del estado del VMA**.

Esto está demostrado empíricamente y es el resultado más fino de las PoC: el contador `denied_mmap` marcó **1 y no 2**. El `mprotect(PROT_WRITE)` devolvió `EACCES` sin que el módulo ejecutara una sola instrucción. La corrección no depende de que hayamos previsto el ataque: depende de una invariante que el kernel comprueba en todas las rutas relevantes, incluidas las que no conocemos.

4.2 El texto plano nunca reside en un proceso de usuario, ni la clave

La clave se entrega como parámetro de módulo con permiso `0`:

```
module_param(key_hex, charp, 0);   /* src/mmapguard.c:85 */
```

Permiso `0` significa que **no se crea entrada en sysfs**: la clave no es legible desde `/sys/module/mmapguard/parameters/`. Vive en memoria de kernel y ningún proceso de usuario la tiene jamás. En cualquier esquema de los de la sección 3, la clave está necesariamente en el espacio de direcciones de un proceso —la aplicación, el interpositor o el demonio FUSE—.

Y la exposición del texto plano está acotada por diseño, no por la política de reclamo del kernel: se descifra una página de 4 KB cuando se toca, y solo esa. Medido: un cliente que lee 64 bytes de un archivo de 2 páginas deja **4.096 bytes en claro en RAM, no 8.192**. Con `remap_pfn_range()` —la técnica del material de referencia— estarían las dos desde el instante del `mmap()`.

4.3 Granularidad de página en el camino del fallo

Es la unión de las dos anteriores: el único lugar desde el cual se puede decidir por página y en el momento del acceso es el camino del fallo, y ese camino está en Ring 0 por definición arquitectónica. No es una preferencia de implementación; es dónde el hardware entrega el control.

---

5. El costo de la mediación completa, cuantificado

La sección 2 explica por qué hay que estar en el camino del fallo. La medición explica lo que eso cuesta, y el dato más interesante es la **simetría con FUSE**.

Nótese que `fuse_file_vm_ops` incluye `.map_pages = filemap_map_pages`. FUSE obtiene fault-around gratis —16 páginas por fallo, `fault_around_pages = 65536 >> PAGE_SHIFT` en `mm/memory.c:5591` — precisamente **porque delega los fallos a la caché de páginas**. Y delegar los fallos a la caché es exactamente lo que le cuesta la mediación por acceso.

Ahí está el intercambio, en una frase:

> FUSE compra fault-around a cambio de renunciar a mediar cada acceso. `mmapguard` compra la mediación de cada acceso a cambio de renunciar al fault-around.

Y eso es, literalmente, el 17,60x: `mmap()` ordinario resuelve 4.097 páginas con 257 fallos; nosotros con 4.097. La razón 15,9x explica casi todo el sobrecosto. **No estamos pagando por AES: estamos pagando por mediar.**

La confirmación es que el costo es proporcional a la mediación que realmente se consume, no un impuesto fijo. Con acceso disperso de salto 64, donde ambos esquemas hacen 65 fallos, el sobrecosto cae a **1,94x** y solo se descifra el 1,6% del archivo. El mecanismo cobra por página mediada.

Queda además margen sin perder nada: implementar `.map_pages` descifrando un lote de páginas por fallo bajaría el secuencial a ~9x conservando la decisión en Ring 0 y la granularidad de página.

---

6. Lo que el espacio de usuario hace mejor

Una comparación en la que una de las partes no tiene ninguna ventaja no es una comparación, es un argumento de venta. Estas son reales y pesan.

Dimensión	Ventaja del espacio de usuario
Aislamiento de fallos	Un error en un demonio FUSE mata un proceso. Un error en `mmapguard.ko` es un oops o un panic: se lleva el sistema. Es la desventaja más grave y no tiene mitigación dentro del enfoque.
Portabilidad	El episodio completo del CRC de `MODVERSIONS` es la evidencia: el módulo está atado a una versión de kernel **y a un compilador** (GCC 13.2). Un binario FUSE cruza versiones sin recompilar.
Tamaño de la base confiable	Añadir ~600 líneas al TCB, en C, sin red de seguridad, es un costo de seguridad real que hay que descontar del beneficio.
Costo de desarrollo	Sin libc, sin coma flotante, pila limitada, contextos de asignación restringidos, depuración por `dmesg`.
Gestión de claves	HSM, PKCS#11, un KMS en red: todo eso es natural en usuario y penoso en un módulo.

Esto es también lo que explica por qué el plano 2 de la arquitectura es **eBPF LSM** y no más código de módulo. eBPF es la síntesis moderna de las dos columnas: mediación en Ring 0, en los ganchos LSM, pero con un **verificador** que elimina el riesgo de panic y con política recargable sin tocar el kernel. Es el enfoque correcto para la cobertura global, y quedó condicionado a un requisito de arranque que este entorno no satisface (`CONFIG_LSM` no incluye `bpf`).

---

7. En qué sentido la capa del kernel es óptima

No en sentido absoluto. La afirmación defendible es condicional, y estas son sus condiciones:

1. **El evento a mediar es el acceso a una página, no la petición de mapeo.** Si bastara con mediar el alta del mapeo, `seccomp` o FUSE serían preferibles por aislamiento de fallos y portabilidad.
2. **El modelo de amenaza incluye procesos del mismo UID, con `ptrace` y `/proc/pid/mem`.** Si el adversario fuera externo al sistema, cifrar en reposo en la aplicación bastaría.
3. **La exposición del texto plano en RAM es parte de lo que hay que acotar.** Es el requisito que descarta a FUSE y a `remap_pfn_range()`, y el que convierte la paginación por demanda de optimización en control de seguridad.
4. **Se acepta pagar el sobrecosto proporcional a las páginas tocadas.** 17,60x secuencial, 1,94x disperso. Para un archivo de configuración con secretos, leído una vez al arrancar y por partes, es irrelevante. Para un flujo secuencial de gran volumen, es inaceptable y la respuesta correcta sería otra: cifrado a nivel de bloque, o `fscrypt`.

Bajo esas cuatro condiciones —que son las del enunciado— la intervención en la capa del kernel no es la mejor de varias opciones viables: es **la única que satisface la mediación completa**. Las demás no pierden por margen, pierden por construcción, porque median una frontera que el evento protegido no cruza.

mmapguard

Seguridad de archivos mediante intervención en la capa del kernel y mapeo de memoria (`mmap`)

Módulo de kernel para Linux que expone archivos cifrados a través de un mapeo de memoria y los descifra **una página de 4 KB a la vez, en el manejador de fallos de página**, de modo que el texto plano nunca existe completo en memoria y ningún proceso de usuario tiene acceso a la clave.

> Parcial 25% — Sistemas Operativos, EAFIT 2026-2. Modalidad individual. El repositorio del curso (`SO2026B/USOS/CapaKernel`) es material de referencia, no la entrega.

⚠ Antes de nada: el kernel tiene que ser el 6.18.33.2

Este módulo solo se carga en `6.18.33.2-microsoft-standard-WSL2`, y solo si se compiló con `gcc-13`. No es una preferencia del autor: con `CONFIG_MODVERSIONS=y`, basta que el vermagic o un único CRC de símbolo no coincidan para que `insmod` falle con `Invalid module format` y el módulo no llegue a ejecutar ni una instrucción.

```
uname -r        # tiene que decir: 6.18.33.2-microsoft-standard-WSL2
```

Si dice otra cosa, el módulo no cargará, y lo más probable es que diga otra cosa: WSL actualiza su kernel por su cuenta con Windows Update. Pasó durante el desarrollo —Windows subió de 6.18.33.2 a 6.18.58.1 de un día para otro y `insmod` empezó a fallar sin que cambiara una línea de código—. En Requisitos está cómo recompilar contra el kernel en ejecución o cómo fijar WSL al 6.18.33.2.

Qué garantiza, con precisión

Propiedad	Mecanismo
El archivo es ilegible en reposo	AES-128-CTR, nonce derivado del índice de página
Solo un UID autorizado puede asociar y mapear	`mg_caller_is_authorized()` en el `ioctl` y en `mmap`
El mapeo es de **solo lectura**, sin excepción	`mg_mmap()` rechaza `PROT_WRITE` y `MAP_SHARED`
`mprotect(PROT_WRITE)` posterior falla	`vm_flags_clear(vma, VM_MAYWRITE)`: lo rechaza **el núcleo**
La exposición en RAM se limita a lo que se toca	`vm_ops->fault`, descifrado bajo demanda
La clave no es accesible desde el espacio de usuario	`module_param(key_hex, charp, 0)`: sin entrada en sysfs
Todo intento queda registrado	`struct mg_stats` vía `ioctl` + `pr_warn` a `dmesg`

Lo que NO hace, dicho sin rodeos:

- **No admite escritura.** No es una limitación de tiempo sino de diseño: el modo CTR reutilizaría clave y nonce al recifrar una página modificada, revelando el XOR de los textos pla-

nos. Admitir escritura exige un modo AEAD con contador de versión. Está especificado como trabajo futuro.

- **Solo protege lo que pasa por `/dev/mmapguard`**, no archivos arbitrarios del sistema de archivos. Extenderlo a cobertura global es el plano 2 de la arquitectura (eBPF LSM), que **no está implementado** por un requisito de arranque que este entorno no satisface.
- **Cuesta 17,60x** en acceso secuencial completo, y 1,94x en acceso disperso. La causa está medida y atribuida más abajo; no es el cifrado.

Requisitos

Este módulo se compiló y probó contra **un único entorno**, y los dos primeros requisitos no son negociables:

Requisito	Valor	Por qué
Kernel	`6.18.33.2-microsoft-standard-WSL2`	`CONFIG_MODVERSIONS=y`: el vermagic y los CRC deben coincidir
Compilador	**`gcc-13`**	El kernel lo compiló Microsoft con GCC 13.2; con GCC 15 los CRC no coinciden (ver abajo)
Árbol de fuentes	compilado por completo, con `Module.symvers`	`modules_prepare` no basta si hay `MODVERSIONS`
Distribución de prueba	Ubuntu 26.04 LTS sobre WSL2	—

```
sudo apt install -y build-essential bison flex libssl-dev libelf-dev cpio \
                    dwarves gcc-13 gcc-13-plugin-dev
```

> **Si `insmod` falla con `Invalid module format`**, consulte `dmesg`. Si dice "disagrees about version of symbol module_layout", se compiló con el compilador equivocado: use `CC=gcc-13`. Si el vermagic lleva un sufijo `+` que el kernel en ejecución no tiene, el árbol de fuentes tiene un directorio `.git` y `scripts/setlocalversion` lo está añadiendo; hay que ocultarlo. El truco de `touch .scmversion` **ya no funciona** en 6.18.

Si su kernel no es el 6.18.33.2

Compruébelo con `uname -r`. Si no coincide hay dos caminos, y el primero es mucho más corto.

Camino corto — recompilar contra el kernel en ejecución. No hace falta el 6.18.33.2 para que el código funcione: hace falta que el módulo y el kernel cargado provengan del mismo árbol, de la misma versión y del mismo compilador. Consiga el árbol de su kernel de WSL, compílelo entero (`make -j$(nproc) CC=gcc-13`, porque `MODVERSIONS` exige `Module.symvers` y `modules_prepare` no lo genera) y apunte `KDIR` ahí. Las PoC pasan igual; lo que caduca son las citas `archivo:línea`.

Camino largo — arrancar WSL con el 6.18.33.2, que es lo que se hizo aquí. Compile ese árbol por completo, lo que produce `arch/x86/boot/bzImage`, cópielo a un sitio visible desde Windows (p. ej. `C:\Users\<usuario>\bzImage-6.18.33.2`) y cree `C:\Users\<usuario>\.wslconfig` con este contenido, **con las barras duplicadas**, que el formato lo exige:

```
[wsl2]
kernel=C:\\Users\\<usuario>\\bzImage-6.18.33.2
```

Después, `wsl --shutdown` en PowerShell y vuelva a entrar. Para revertir, borre esa línea y repita el `wsl --shutdown`.

Un aviso antes de fijarlo: **si ese `.config` no trae compilados dentro los controladores del disco raíz, WSL no arranca.** En este entorno la raíz es `/dev/sdd`, de modo que hacían falta `CONFIG_SCSI=y`, `BLK_DEV_SD=y` y `SCSI_VIRTIO=y`, además de `HYPERV`, `VSOCKETS` y `CRYPTO_CTR`. Compruébelos con `grep` sobre el `.config` antes de reiniciar.

Por qué la versión importa más allá de poder cargar el módulo: todas las citas `archivo:línea` del informe y todas las cifras de sobrecosto se tomaron contra el árbol 6.18.33.2. Cambiar de kernel obliga a reverificarlas una por una.

Compilación

```
# 1. El módulo de kernel
make modules KDIR=/ruta/al/arbol/del/kernel CC=gcc-13

# 2. Las herramientas de espacio de usuario
make tools

# 3. Los datos de demostración (se generan, no vienen en el repositorio)
make demo
```

`make help` lista todos los objetivos. El módulo compila **sin una sola advertencia** con `-Wall -Wextra`, que es un requisito del enunciado.

Uso

Las operaciones privilegiadas van marcadas con `sudo`.

```
# Cargar, autorizando al UID 1000
sudo insmod mmapguard.ko authorized_uid=1000
sudo chmod 666 /dev/mmapguard

# Ejecutar el banco completo de pruebas de concepto
./tests/run_pocs.sh                # con el modulo ya cargado
make test TESTFLAGS=--load         # que el script lo cargue y lo descargue

# O solo el cliente de referencia en C
cd demo && ../tools/mg_test secreto.enc

# Medir el sobrecosto (stride: 1 = secuencial, 8 = salto medio, 64 = disperso)
../tools/mg_bench grande.txt grande.enc 1

# Descargar
sudo rmmod mmapguard
```

Parámetros del módulo

Parámetro	Valor por omisión	Significado
`authorized_uid`	`-1` (solo root)	UID efectivo autorizado a asociar y mapear
`key_hex`	clave de demostración	Clave AES-128 en hexadecimal, 32 caracteres

Para cifrar un archivo propio con el esquema que el driver espera:

```
tools/mg_encrypt claro.txt cifrado.enc 00112233445566778899aabbccddeeff
sudo rmmod mmapguard && sudo insmod mmapguard.ko \
     authorized_uid=1000 key_hex=00112233445566778899aabbccddeeff
```

> ⚠ La clave `000102030405060708090a0b0c0d0e0f` que trae el código es una **clave de demostración** para que las pruebas corran sin configuración. No tiene otro propósito. Nótese además que `key_hex` se declara con permiso `0`, de modo que no queda legible en `/sys/module/mmapguard/parameters/`, pero sí es visible en la línea de `insmod` y por tanto en el historial del intérprete de comandos: en un uso real la clave debe entrar por otra vía.

Pruebas de concepto

`mg_test` ejecuta tres ataques y comprueba que el mecanismo los ataja. Salida esperada:

```
=== mmapguard: prueba del plano de datos ===

[A1] Lectura DIRECTA del archivo (sin el driver):
     <bytes ilegibles>
     -> ilegible: el archivo esta cifrado en reposo.

[+] Archivo asociado al driver.

[  ] Lectura a traves del MAPEO PROTEGIDO:
     NOMINA CONFIDENCIAL - EAFIT 2026-2 - NO DIVULGAR
     -> legible: descifrado pagina a pagina en el fallo.

[A3] Intento de escalada con mprotect(PROT_WRITE):
     -> DENEGADO (Permission denied): VM_MAYWRITE fue retirado en el mmap.

[A2] Intento de mmap con PROT_WRITE:
     -> DENEGADO (Permission denied): politica de solo lectura.

--- Auditoria del driver ---
  Tamano del archivo    : 4764 bytes
  Paginas descifradas   : 1
  Bytes descifrados     : 4096
  Mapeos denegados      : 1
  Escrituras denegadas  : 0
```

Las dos últimas cifras son el resultado más informativo del conjunto:

- **`Mapeos denegados: 1`, no 2.** Hubo dos ataques de escritura y el módulo solo contó uno: el `mprotect` **nunca llegó al módulo**. Lo rechazó el propio núcleo al comprobar `VM_MAYWRITE`. Eso es precisamente el efecto buscado al retirar ese bit en vez de interceptar la llamada: la política es un invariante del VMA, no una lista de comprobaciones que haya que acertar a enumerar.
- **`Escrituras denegadas: 0`.** No es un mecanismo que falló, es la medida de que la ruta de escritura es **inalcanzable**: `page_mkwrite` está registrado como defensa en profundidad y nunca tuvo que actuar.

El banco completo: `tests/run_pocs.sh`

`mg_test` es el cliente de referencia en C y cubre A1, A2 y A3 en una sola sesión. El banco de pruebas lo envuelve y añade las PoC que necesitan control crudo de las banderas de `mmap`, de `mprotect` y de `madvise`, más el errno exacto de cada rechazo:

```
./tests/run_pocs.sh --help
./tests/run_pocs.sh --load          # carga el modulo, prueba, descarga
./tests/run_pocs.sh --quick         # acorta la fase de martilleo de A5
./tests/run_pocs.sh --file demo/grande.enc --claro demo/grande.txt
```

PoC	Ataque	Resultado medido (`tests/last-run.log` , 8/8 PASS)	Pregunta
A0	Asociar un archivo desde un UID no autorizado	`EPERM` y `denied_mmap` se incrementa	4
A1	`read()` directo sobre el archivo	Ilegible en reposo	5
OK	Lectura por el mapeo protegido	Los bytes coinciden **completos** con el claro	1, 5
A2	`mmap` con `PROT_WRITE` , y con `MAP_SHARED`	`EACCES` ; con `MAP_SHARED` sobre un `fd` de solo lectura, el núcleo degrada el VMA antes de llegar al módulo	2
A3	`mprotect(PROT_WRITE)` tras un mapeo concedido	`EACCES` , y `denied_mmap` **no** cambia	3
A4	`fd` en `O_RDONLY` mapeado `PROT_WRITE|MAP_PRIVATE`	Permitido sobre un archivo ordinario, denegado sobre el dispositivo	8
A5	`madvise(MADV_DONTNEED)` martilleado contra escritura	`SIGSEGV` ; 21.615 descartes, +6.665 fallos de página, 0 discrepancias, 0 bytes redescifrados	9

Dos de ellas merecen una nota, porque no son simples repeticiones del patrón «se intenta, se deniega»:

- **A4 demuestra primero que la evasión existe.** Sobre `demo/secreto.txt` , abierto en `O_RDONLY` , el núcleo **concede** el mapeo `PROT_WRITE | MAP_PRIVATE` ; la PoC altera los primeros ocho bytes en memoria (`NOMINA C` → `AAAAAAAA`) y comprueba por `sha256` que el archivo en disco no cambió. Esa es exactamente la situación que describe la pregunta 8: `O_RDONLY` no impide ni ver el contenido en claro ni alterarlo en la copia privada. Solo después se repite el mismo ataque contra `/dev/mmapguard` , donde `mg_mmap()` lo rechaza.
- **A5 no se conforma con argumentar que no hay carrera.** La fase 1 lanza un proceso hijo que intenta escribir en el mapeo y verifica que muere con `SIGSEGV` — el proceso hijo está aislado precisamente porque esa señal no se puede capturar, y su muerte **es** el resultado positivo. La fase 2 martillea `MADV_DONTNEED` desde cuatro hilos mientras el hilo principal reverifica el digest del mapeo completo, y mide `ru_minflt` con `getrusage()` para probar que las PTE se estaban descartando de verdad: sin ese dato, un martilleo que no descarta nada «pasaría» la prueba sin haber probado nada. Por eso el banco distingue `INCONCLUSIVE` de `PASS` .

A0 necesita `sudo` y `setpriv` , porque ejecutar bajo otro UID es una operación privilegiada. Además copia `mg_poc.py` y el archivo cifrado a un directorio temporal de permisos `755` antes de bajar de privilegios: con un directorio personal `drwxr-x---` —el valor habitual— el UID de prueba no puede ni atravesar la ruta hasta el script, y la PoC fallaría sin llegar a ejercitar nada. Se prepara una copia accesible en vez de relajar los permisos del directorio personal.

Cada PoC devuelve un código de salida propio — `0` PASS, `1` FAIL, `2` INCONCLUSIVE, `3` problema de entorno — y el orquestador los agrega, de modo que el banco sirve igual para la demostración en vivo que para integración continua. El script también verifica dos invariantes del entorno antes de empezar: que `authorized_uid` coincida con el UID que ejecuta (para no confundir una política con una mala configuración) y que `key_hex` **no** aparezca en `/sys/module/mmapguard/parameters/` .

Mapeo de cada PoC con las preguntas de la Parte I: **A0 → 4, A1 → 5, A2 → 2, A3 → 3, A4 → 8, A5 → 9.** A6 (`msync`, pregunta 6) no aplica a un mapeo de solo lectura: exige el modo AEAD que se documenta como trabajo futuro.

Sobrecosto medido

Archivo de 16.777.285 bytes (4.097 páginas), mediana de 11 repeticiones.

Patrón de acceso	Páginas tocadas	Fallos: `mmap` / `mmapguard`	Sobrecosto	Archivo descifrado
Secuencial	4.097	257 / 4.097	**17,60x**	100 %
Salto de 8	513	257 / 513	2,93x	12,5 %
Disperso (64)	65	65 / 65	**1,94x**	**1,6 %**

El costo dominante no es AES, es la ausencia de fault-around. `mmap()` ordinario puebla 16 PTE por fallo (`fault_around_pages = 65536 >> PAGE_SHIFT`); nosotros una. La razón 15,9x explica casi todo el 17,60x, y la prueba es la última fila: cuando ambos esquemas hacen los mismos 65 fallos, el sobrecosto cae a 1,94x. El mecanismo cobra por página mediada, no como impuesto fijo.

Implementar `.map_pages` —descifrar un lote de páginas por fallo— bajaría el secuencial a ~9x sin perder ni la decisión en Ring 0 ni la granularidad de página. Está identificado y no implementado.

Arquitectura

Un plano por cada objetivo del enunciado:

Objetivo	Plano	Mecanismo	Estado
Aislamiento de memoria	Datos	`vm_ops->fault` con descifrado por página	**Implementado**
Prevención de modificaciones	Política	`mg_mmap()` + `vm_flags_clear(VM_MAYWRITE)`	**Implementado** (alcance: `/dev/mmapguard`)
		eBPF LSM sobre `mmap_file` y `file_mprotect`	No implementado (alcance global)
Auditoría de accesos	Observabilidad	`struct mg_stats` vía `ioctl` + `getrusage`	**Implementado**

La decisión técnica central es usar `vm_operations_struct->fault` en lugar de `remap_pfn_range()`, que es lo que emplea el material de referencia del curso. `remap_pfn_range()` programa todas las PTE de golpe, lo que obliga a mantener **el archivo completo descifrado en RAM** durante toda la vida del mapeo. Con el manejador de fallos se descifra una página cuando se toca, y solo esa: un cliente que lee 64 bytes de un archivo de 2 páginas deja 4.096 bytes en claro, no 8.192. Eso convierte la paginación por demanda, que en el material de referencia es una optimización, en un **control de minimización de exposición**.

Alternativas descartadas, cada una por una razón verificada contra el árbol de fuentes: hook en la tabla de syscalls (exige desarmar el bit WP de `CR0`), LSM propio como módulo car-

gable (`security_add_hooks()` no está exportada), y eBPF solo con `fentry` / `kprobes` (observa pero no puede denegar).

Por qué en el kernel y no en espacio de usuario. Porque el evento que hay que mediar —el acceso a una página— **no es una llamada al sistema**, y todo lo que el espacio de usuario puede interceptar es una frontera de syscall. `strace` sobre el cliente registra 30 llamadas a `mmap` y ninguna señal de las 4.097 páginas descifradas. El argumento completo, incluido el examen de `seccomp` y de FUSE, está en docs/04-justificacion-kernel-vs-usuario.md.

Estructura

```
mmapguard/
├── Makefile                  # modulo + objetivos tools, demo, test, clean, help
├── src/
│   ├── mmapguard.c           # el modulo
│   └── mmapguard_uapi.h      # contrato ioctl compartido con el usuario
├── tools/
│   ├── mg_encrypt.c          # cifra un archivo con el esquema del driver (OpenSSL)
│   ├── mg_test.c             # cliente de referencia en C: A1, A2 y A3
│   └── mg_bench.c            # comparativa a 3 bandas con getrusage
├── tests/
│   ├── run_pocs.sh           # orquestador: entorno, ejecucion, dmesg, resumen
│   ├── mg_poc.py             # las PoC A0-A5 por ctypes (mmap/mprotect/madvise)
│   └── last-run.log          # salida de la ultima ejecucion (no se versiona)
├── demo/
│   └── gen_demo.sh           # genera los datos de prueba, deterministas
└── docs/
    ├── 02-preguntas.md                       # Parte I: 10 preguntas conceptuales
    ├── 03-1-flujo-trabajo-ia.md              # 3.1 flujo de trabajo asistido por IA
    ├── 03-2-prompts-validacion.md            # 3.2 prompts y alucinaciones detectadas
    ├── 03-3-arquitectura.md                  # 3.3 estrategia arquitectonica
    ├── arquitectura.svg                      # el diagrama, dibujado para impresion
    ├── breve/                                # los mismos documentos, condensados
    ├── build_report.py                       # ensambla docs/ + README + log -> los PDF
    ├── 04-1-implementacion.md                # 4.1 decisiones de implementacion
    ├── 04-resultados.md                      # 4.2 medicion de impacto
    └── 04-justificacion-kernel-vs-usuario.md  # 4.2 justificacion teorica
```

Los dos PDF del informe se construyen con `make informes PYTHON=<venv>/bin/python INFORMEFLAGS='--autor "Nombre"'`:

Archivo	Págs.	Fuente	Para qué
`reporte parcial 3 resumen.pdf`	20	`docs/breve/`	**El informe que se entrega**
`reporte parcial 3 completo.pdf`	63	`docs/`	Versión íntegra, con este README y `last-run.log` como anexos

Los datos de `demo/` y los artefactos de compilación no se versionan: se regeneran con `make demo`, que produce archivos **byte a byte idénticos** a los usados en las mediciones de este informe.

Notas de reproducibilidad

Las cifras de este README salieron de un único entorno y dependen de él. Dos advertencias metodológicas que vale la pena heredar:

1. **Compile las mediciones de forma que el optimizador no las borre.** El banco de pruebas del repositorio de referencia (`paso5_benchmark_comparativo`) nunca consume el checksum de su bucle, así que GCC `-O2` lo elimina por código muerto y reporta 3.134.335 MB/s y 0 fallos de página para 64 MB: ambas cifras imposibles. Recompilado con `-O0` da 11.552 MB/s y 1.024 fallos. `mg_bench` imprime todos sus checksums precisamente por eso.
2. **No confíe en una sola muestra.** La primera medición de latencia de `mmap()` dio 43,2 µs frente a 3,2 µs; con 11 repeticiones y mediana, la diferencia quedó por debajo del ruido del planificador.

Y una tercera, sobre el instrumento: **`strace` es inútil para observar este módulo**, porque un fallo de página no es una syscall. Hay que usar `getrusage`, `ftrace` o `perf`.

Anexo B. Registro del banco de pruebas

Salida íntegra de `./tests/run_pocs.sh --load`, incluido el registro del núcleo que el módulo emitió durante la ejecución.

```
=== mmapguard: banco de pruebas de concepto ===
    kernel        : 6.18.33.2-microsoft-standard-WSL2
    usuario       : uid=1000 (sebas)
    archivo cifrado: /home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/demo/secreto.enc
    texto en claro : /home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/demo/secreto.txt

[*] Cargando mmapguard.ko con authorized_uid=1000 (requiere sudo)

    authorized_uid del modulo: 1000
    clave en sysfs: no expuesta (correcto)

--- A0  Asociacion desde un UID no autorizado (uid=65534)
    MG_IOC_ATTACH rechazado: Operation not permitted (errno 1/EPERM)
    contadores: faults=0 bytes_descifrados=0 denied_mmap=1 denied_write=0
    -> PASS: el driver rechaza el UID no autorizado y lo audita

--- A1  Lectura directa del archivo con read(), sin pasar por el driver
    primeros 64 bytes: ..v~..{. ..+...::._....._.2..W..<d..~......,=,.....F^X..\?qu?...z
    fraccion imprimible: 0.41 (texto en claro ~1.00)
    difiere del texto en claro de referencia
    -> PASS: ilegible en reposo: el cifrado protege el archivo en disco

--- OK  Lectura a traves del mapeo protegido (PROT_READ, MAP_PRIVATE)
    primeros 64 bytes: NOMINA CONFIDENCIAL - EAFIT 2026-2 - NO DIVULGAR.Empleado,Cargo,
    contadores: faults=1 bytes_descifrados=4096 denied_mmap=0 denied_write=0
    los 4764 bytes del mapeo coinciden con '/home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/
demo/secreto.txt' (sha256 b74067de563fd176...)
    nota: se materializaron 1 paginas de las 2 que cubre el archivo
    -> PASS: el mapeo entrega el texto en claro, descifrado por pagina

--- A2  mmap() con PROT_WRITE, y mmap() con MAP_SHARED
    PROT_READ|PROT_WRITE, MAP_PRIVATE: DENEGADO con Permission denied (errno 13/EACCES)
    PROT_READ, MAP_SHARED (fd O_RDONLY): concedido; maps dice 'r--s'
        smaps VmFlags: dd de me mr ms rd sd
        sin 'sh' ni 'mw': el nucleo retiro VM_SHARED y VM_MAYWRITE porque el fd no tiene FMODE_WRITE (mm/
mmap.c:462-464)
        maps sigue mostrando 's' porque ese caracter depende de VM_MAYSHARE, no de VM_SHARED (fs/proc/
task_mmu.c:454)
        mprotect(PROT_WRITE) sobre el: DENEGADO con Permission denied (errno 13/EACCES); VM_MAYWRITE tambien se
retiro
    contadores: faults=0 bytes_descifrados=0 denied_mmap=1 denied_write=0
    PROT_READ, MAP_SHARED (fd O_RDWR): DENEGADO con Permission denied (errno 13/EACCES)
    contadores: faults=0 bytes_descifrados=0 denied_mmap=1 denied_write=0
    -> PASS: PROT_WRITE lo ataja mg_mmap(); MAP_SHARED lo degrada el nucleo con un fd de solo lectura y lo ata-
ja mg_mmap() con uno escribible

--- A3  Escalada de privilegios con mprotect(PROT_WRITE)
    mprotect(PROT_WRITE) DENEGADO: Permission denied (errno 13/EACCES)
    contadores: faults=1 bytes_descifrados=4096 denied_mmap=0 denied_write=0
    denied_mmap no cambio: la llamada no llego al modulo, la rechazo el nucleo al ver que falta VM_MAYWRITE
    -> PASS: el nucleo cierra la escalada sin interceptar mprotect

--- A4  fd en O_RDONLY mapeado PROT_WRITE|MAP_PRIVATE
    sobre 'secreto.txt' (O_RDONLY): el nucleo CONCEDE el mapeo escribible
        en memoria: b'NOMINA C' -> b'AAAAAAAA'
    el archivo en disco NO cambio (sha256 identico): la alteracion vive solo en la copia COW del proceso
    esa es la evasion que describe la pregunta 8: O_RDONLY no impide alterar los datos en memoria ni verlos en
claro
    sobre /dev/mmapguard: DENEGADO con Permission denied (errno 13/EACCES)
    contadores: faults=0 bytes_descifrados=0 denied_mmap=1 denied_write=0
    -> PASS: la evasion existe en el caso general y mg_mmap() la cierra para los archivos protegidos

--- A5  Carrera entre madvise(MADV_DONTNEED) y escritura
    fase 1: un proceso hijo intenta escribir en el mapeo protegido
        el hijo murio con SIGSEGV: la escritura la rechaza la MMU por el bit R/W de la PTE, sin llegar al modu-
lo
    fase 2: 4 hilos martilleando MADV_DONTNEED durante 2s mientras se relee el mapeo
        descartes: 21615  relecturas completas: 11229  discrepancias: 0
        fallos de pagina menores del proceso: +6665
        contadores del driver: faults +0, bytes descifrados +0
    el driver no redescifro: el descarte retira la PTE pero la pagina en claro sigue en la cache de la sesion
    -> PASS: sin escritura posible no hay carrera; el descarte repetido no altera ni corrompe el contenido

--- REF  Cliente de referencia en C (tools/mg_test)
=== mmapguard: prueba del plano de datos ===

[A1] Lectura DIRECTA del archivo (sin el driver):
     ..v~..{. ..+...::._....._.2..W..<d..~......,=,.....F^X..\?qu?...z
     -> ilegible: el archivo esta cifrado en reposo.

[+] Archivo asociado al driver.
```

```
[  ] Lectura a traves del MAPEO PROTEGIDO:
     NOMINA CONFIDENCIAL - EAFIT 2026-2 - NO DIVULGAR.Empleado,Cargo,
     -> legible: descifrado pagina a pagina en el fallo.

[A3] Intento de escalada con mprotect(PROT_WRITE):
     -> DENEGADO (Permission denied): VM_MAYWRITE fue retirado en el mmap.

[A2] Intento de mmap con PROT_WRITE:
     -> DENEGADO (Permission denied): politica de solo lectura.

--- Auditoria del driver ---
  Tamano del archivo    : 4764 bytes
  Paginas descifradas   : 1
  Bytes descifrados     : 4096
  Mapeos denegados      : 1
  Escrituras denegadas  : 0

--- Registro del kernel (lo que el modulo audito) ---
    [ 1308.630820] mmapguard: mapeo concedido [0x762e44c4b000-0x762e44c4d000] 8192 bytes, solo lectura (PID
1748)
    [ 1308.630844] mmapguard: pagina 0 descifrada bajo demanda (4096 bytes) por PID 1748
    [ 1308.630894] mmapguard: cerrando sesion (fallos=1, descifrados=4096 bytes, mmap denegados=0, escrituras
denegadas=0)
    [ 1308.698950] mmapguard: asociado '/home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/
demo/secreto.enc' (4764 bytes, 2 paginas) por PID 1749
    [ 1308.698969] mmapguard: MMAP DENEGADO: se solicito PROT_WRITE (PID 1749)
    [ 1308.699008] mmapguard: cerrando sesion (fallos=0, descifrados=0 bytes, mmap denegados=1, escrituras de-
negadas=0)
    [ 1308.767848] mmapguard: asociado '/home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/
demo/secreto.enc' (4764 bytes, 2 paginas) por PID 1751
    [ 1308.767934] mmapguard: mapeo concedido [0x708e13e34000-0x708e13e36000] 8192 bytes, solo lectura (PID
1751)
    [ 1308.767980] mmapguard: pagina 0 descifrada bajo demanda (4096 bytes) por PID 1751
    [ 1308.768801] mmapguard: cerrando sesion (fallos=1, descifrados=4096 bytes, mmap denegados=0, escrituras
denegadas=0)
    [ 1308.769200] mmapguard: asociado '/home/sebas/proyectos/universidad/OS/git_profe/parcial_3/mmapguard/
demo/secreto.enc' (4764 bytes, 2 paginas) por PID 1750
    [ 1308.769263] mmapguard: mapeo concedido [0x708e13e34000-0x708e13e36000] 8192 bytes, solo lectura (PID
1750)
    [ 1308.769301] mmapguard: pagina 0 descifrada bajo demanda (4096 bytes) por PID 1750
    [ 1308.769306] mmapguard: pagina 1 descifrada bajo demanda (668 bytes) por PID 1750
    [ 1310.774197] mmapguard: cerrando sesion (fallos=2, descifrados=4764 bytes, mmap denegados=0, escrituras
denegadas=0)
    [ 1310.797133] mmapguard: asociado 'secreto.enc' (4764 bytes, 2 paginas) por PID 1756
    [ 1310.797147] mmapguard: mapeo concedido [0x73faf2b19000-0x73faf2b1b000] 8192 bytes, solo lectura (PID
1756)
    [ 1310.797160] mmapguard: pagina 0 descifrada bajo demanda (4096 bytes) por PID 1756
    [ 1310.797189] mmapguard: MMAP DENEGADO: se solicito PROT_WRITE (PID 1756)
    [ 1310.797198] mmapguard: cerrando sesion (fallos=1, descifrados=4096 bytes, mmap denegados=1, escrituras
denegadas=0)

=== Resumen ===
    PRUEBA                      RESULTADO
    A0  UID no autorizado       PASS
    A1  Cifrado en reposo       PASS
    OK  Descifrado correcto     PASS
    A2  mmap PROT_WRITE/SHARED  PASS
    A3  Escalada con mprotect   PASS
    A4  Evasion de O_RDONLY     PASS
    A5  Carrera con madvise     PASS
    REF Cliente en C            PASS

    8/8 PASS, 0 FAIL, 0 INCONCLUSIVE, 0 ENV
[+] Todas las pruebas pasaron.
[*] Descargando mmapguard (lo cargo este script)
```